TUTORIAL DE CURSORES EN PL/pgSQL - PostgreSql 18

utilizando esta base de datos:

```
CREATE SCHEMA tienda;
SET search_path TO tienda;

CREATE TABLE clientes (
    cliente_id   SERIAL PRIMARY KEY,
    nombre       TEXT NOT NULL,
    email        TEXT UNIQUE NOT NULL,
    fecha_alta   DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE productos (
    producto_id  SERIAL PRIMARY KEY,
    nombre       TEXT NOT NULL,
    precio       NUMERIC(10,2) NOT NULL,
    stock        INTEGER NOT NULL DEFAULT 0,
    activo       BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE pedidos (
    pedido_id    SERIAL PRIMARY KEY,
    cliente_id   INTEGER NOT NULL REFERENCES clientes(cliente_id),
    fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado       TEXT NOT NULL DEFAULT 'pendiente'
);

CREATE TABLE lineas_pedido (
    linea_id     SERIAL PRIMARY KEY,
    pedido_id    INTEGER NOT NULL REFERENCES pedidos(pedido_id),
    producto_id  INTEGER NOT NULL REFERENCES productos(producto_id),
    cantidad     INTEGER NOT NULL,
    precio_unit  NUMERIC(10,2) NOT NULL
);

CREATE TABLE pagos (
    pago_id      SERIAL PRIMARY KEY,
    pedido_id    INTEGER NOT NULL REFERENCES pedidos(pedido_id),
    fecha_pago   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    importe      NUMERIC(10,2) NOT NULL,
    metodo       TEXT NOT NULL
);

CREATE TABLE empleados (
```

```
    id SERIAL PRIMARY KEY,
    nombre TEXT NOT NULL,
    salario NUMERIC(10,2) NOT NULL,
    departamento TEXT,
    fecha_contratacion DATE DEFAULT CURRENT_DATE
);

INSERT INTO empleados (nombre, salario, departamento)
VALUES
('Ana Torres',        2800.00, 'Ventas'),
('Luis Martínez',     4500.00, 'IT'),
('Carla Gómez',       5200.00, 'IT'),
('Dmitry Komarov',    3100.00, 'Contabilidad'),
('James Smith',       7000.00, 'Dirección'),
('Mary Sexton',       3900.00, 'Marketing'),
('Agnes Miller',      6100.00, 'IT'),
('Chun Yen',          2950.00, 'Ventas');


INSERT INTO clientes (nombre, email) VALUES
('Juan Pérez', 'juan@example.com'),
('María López', 'maria@example.com');

INSERT INTO productos (nombre, precio, stock) VALUES
('Teclado mecánico', 79.90, 50),
('Ratón inalámbrico', 39.50, 100),
('Monitor 27"', 249.00, 20);

INSERT INTO pedidos (cliente_id, estado) VALUES
(1, 'pendiente'),
(2, 'pendiente');

INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_unit) VALUES
(1, 1, 1, 79.90),
(1, 2, 2, 39.50),
(2, 3, 1, 249.00);

INSERT INTO pagos (pedido_id, importe, metodo) VALUES
(1, 158.90, 'tarjeta'),
(2, 249.00, 'transferencia');
```

Perfecto, Jose. Aquí tienes **EL TUTORIAL DEFINITIVO DE CURSORES EN PL/pgSQL para PostgreSQL 18**, totalmente adaptado a tu base de datos *tienda*, con **ejemplos completos**, **enunciados**, **resultados**, **cursores implícitos, explícitos, dinámicos**, y **funciones con variables OUT y RETURN SETOF RECORD**, usando SIEMPRE tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_cliente_id`, `v_producto_id`, etc.

Incluyo Guided Links para que puedas profundizar en cada concepto cuando quieras:

- **Cursores implícitos**

- **Cursores explícitos**
- **Cursores dinámicos**
- **Variables OUT**
- **Funciones SETOF RECORD**

1. ¿Qué es un cursor en PL/pgSQL?

Un **cursor** es un puntero a un conjunto de resultados. Sirve para **recorrer filas una a una**, procesarlas, devolverlas, o generar reportes complejos.

2. Cursores implícitos

Los cursores implícitos se crean automáticamente con:

sql

```
FOR v_registro IN SELECT ... LOOP
```

No se declaran, no se abren, no se cierran.

Ejemplo 1 — Cursor implícito para recorrer pedidos

Enunciado

Recorrer todos los pedidos y mostrar su total calculado desde las líneas.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_listar_totales_pedidos()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_pedido_id INTEGER;
    v_total NUMERIC(10,2);
BEGIN
    FOR v_pedido_id IN
        SELECT pedido_id FROM tienda.pedidos
    LOOP
        SELECT SUM(cantidad * precio_unit)
        INTO v_total
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id;

        RAISE NOTICE 'Pedido %, Total %', v_pedido_id, v_total;
    END LOOP;
END;
$$;
```

Resultado esperado

Código

```
NOTICE: Pedido 1, Total 158.90
NOTICE: Pedido 2, Total 249.00
```

3. Cursores explícitos

Aquí declaras el cursor, lo abres, haces FETCH, y lo cierras.

Ejemplo 2 — Cursor explícito con variables OUT

Enunciado

Devolver información de cada línea de pedido usando un cursor explícito y variables OUT.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_lineas_pedido(
    OUT v_linea_id INTEGER,
    OUT v_pedido_id INTEGER,
    OUT v_producto_id INTEGER,
    OUT v_cantidad INTEGER,
    OUT v_precio_unit NUMERIC
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;

BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_cursor_lineas_pedido();
```

Código

```
 v_linea_id | v_pedido_id | v_producto_id | v_cantidad | v_precio_unit
------------+-------------+---------------+------------+---------------
     1      |      1      |       1       |     1      |    79.90
     2      |      1      |       2       |     2      |    39.50
     3      |      2      |       3       |     1      |    249.00
```

4. Cursores explícitos con SCROLL

Permiten moverte hacia adelante y atrás.

Ejemplo 3 — Cursor SCROLL

Enunciado

Leer la primera línea, luego saltar a la tercera.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_scroll()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c SCROLL CURSOR FOR
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c;

    FETCH FIRST FROM c INTO v_linea_id, v_cantidad;
    RAISE NOTICE 'Primera línea: %, cantidad %', v_linea_id, v_cantidad;

    FETCH ABSOLUTE 3 FROM c INTO v_linea_id, v_cantidad;
    RAISE NOTICE 'Tercera línea: %, cantidad %', v_linea_id, v_cantidad;

    CLOSE c;
END;
$$;
```

Resultado

Código

```
NOTICE: Primera línea: 1, cantidad 1
NOTICE: Tercera línea: 3, cantidad 1
```

5. Cursores dinámicos

Se abren con `EXECUTE` y permiten SQL dinámico.

Ejemplo 4 — Cursor dinámico con parámetro

Enunciado

Devolver productos con stock mayor a un valor dinámico.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_dinamico_stock(
    p_stock_min INTEGER
)
RETURNS refcursor
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_stock';
BEGIN
    OPEN v_cursor FOR EXECUTE
        'SELECT producto_id, nombre, stock
         FROM tienda.productos
         WHERE stock > ' || p_stock_min;

    RETURN v_cursor;
END;
$$;
```

Uso

sql

```
BEGIN;
SELECT tienda.fn_cursor_dinamico_stock(30);
FETCH ALL FROM cur_stock;
COMMIT;
```

Resultado

Código

```
 producto_id | nombre            | stock
-------------+-------------------+-------
     2       | Ratón inalámbrico | 100
```

6. Cursores + OUT + SQL dinámico

Nivel avanzado.

Ejemplo 5 — Devolver líneas de pedido filtradas dinámicamente

Enunciado

Filtrar líneas por pedido usando SQL dinámico y variables OUT.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_por_pedido(
    p_pedido_id INTEGER,
    OUT v_linea_id INTEGER,
    OUT v_producto_id INTEGER,
    OUT v_cantidad INTEGER,
    OUT v_precio_unit NUMERIC
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_lineas';
BEGIN
    OPEN v_cursor FOR EXECUTE
        'SELECT linea_id, producto_id, cantidad, precio_unit
         FROM tienda.lineas_pedido
         WHERE pedido_id = ' || p_pedido_id;

    LOOP
        FETCH v_cursor INTO
            v_linea_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_lineas_por_pedido(1);
```

Código

```
 v_linea_id | v_producto_id | v_cantidad | v_precio_unit
------------+---------------+------------+---------------
     1      |       1       |     1      |    79.90
     2      |       2       |     2      |    39.50
```

7. Cursores SETOF refcursor (varios cursores)

Devuelves varios cursores desde una sola función.

Ejemplo 6 — Tres cursores: clientes, productos activos, pedidos

Enunciado

Devolver tres cursores distintos.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_varios_cursores()
RETURNS SETOF refcursor
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor;
BEGIN
    v_cursor := 'cur_clientes';
    OPEN v_cursor FOR SELECT * FROM tienda.clientes;
    RETURN NEXT v_cursor;

    v_cursor := 'cur_productos';
    OPEN v_cursor FOR SELECT * FROM tienda.productos WHERE activo = TRUE;
    RETURN NEXT v_cursor;

    v_cursor := 'cur_pedidos';
    OPEN v_cursor FOR SELECT * FROM tienda.pedidos;
    RETURN NEXT v_cursor;

    RETURN;
END;
$$;
```

Uso

sql

```
BEGIN;
SELECT * FROM tienda.fn_varios_cursores();

FETCH ALL FROM cur_clientes;
FETCH ALL FROM cur_productos;
FETCH ALL FROM cur_pedidos;
COMMIT;
```

8. Cursores implícitos + OUT (combinado)

Enunciado

Recorrer pedidos y devolverlos con OUT.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_out(
    OUT v_pedido_id INTEGER,
    OUT v_cliente_id INTEGER,
    OUT v_estado TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_pedido_id, v_cliente_id, v_estado IN
        SELECT pedido_id, cliente_id, estado
        FROM tienda.pedidos
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_pedidos_out();
```

¿Qué quieres que prepare ahora?

Puedo crear:

- **Cursores con transacciones largas**
- **Cursores para ETL y batch jobs**
- **Cursores con RETURN TABLE**
- **Cursores con manejo de errores**

Dime qué módulo quieres que prepare y te lo construyo con ejemplos completos y tu formato de variables.

return table postgresql
Perfecto, Jose. Vamos a crear **EL MÓDULO COMPLETO DE RETURN TABLE en PL/pgSQL para PostgreSQL 18**, con ejemplos avanzados, claros, y usando SIEMPRE tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc.

Incluyo Guided Links para que puedas profundizar cuando quieras:

- **Return Table**

- **Funciones SETOF RECORD**
- **Cursores explícitos**
- **Cursores implícitos**

1. ¿Qué es RETURN TABLE en PL/pgSQL?

`RETURN TABLE` permite definir columnas de salida directamente en la firma de la función, sin necesidad de declarar variables OUT una por una.

Ejemplo de firma:

sql

```
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER
)
```

Dentro del cuerpo, simplemente asignas valores a esas variables y usas `RETURN NEXT`.

Ejemplo 1 — RETURN TABLE con cursor implícito

Enunciado

Listar todas las líneas de pedido usando un cursor implícito y devolverlas con `RETURN TABLE`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_return_table()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        RETURN NEXT;
```

```
    END LOOP;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_lineas_return_table();
```

2. RETURN TABLE + Cursor explícito

Enunciado

Recorrer los productos activos con un cursor explícito y devolverlos con RETURN TABLE.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_activos_return_table()
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC,
    v_stock INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_productos CURSOR FOR
        SELECT producto_id, nombre, precio, stock
        FROM tienda.productos
        WHERE activo = TRUE
        ORDER BY producto_id;
BEGIN
    OPEN c_productos;

    LOOP
        FETCH c_productos INTO
            v_producto_id,
            v_nombre,
            v_precio,
            v_stock;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE c_productos;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_productos_activos_return_table();
```

3. RETURN TABLE + SQL dinámico

Enunciado

Filtrar productos por stock mínimo usando SQL dinámico y devolverlos con `RETURN TABLE`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_stock_min_return_table(
    p_stock_min INTEGER
)
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC,
    v_stock INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_prod';
BEGIN
    v_sql := format(
        'SELECT producto_id, nombre, precio, stock
         FROM tienda.productos
         WHERE stock >= %s
         ORDER BY producto_id',
        p_stock_min
    );

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO
            v_producto_id,
            v_nombre,
            v_precio,
            v_stock;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_productos_stock_min_return_table(30);
```

4. RETURN TABLE + JOIN + lógica adicional

Enunciado

Devolver un reporte completo de líneas de pedido con nombre del producto y total calculado.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_reporte_lineas_return_table()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_nombre_producto TEXT,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id,
        v_pedido_id,
        v_producto_id,
        v_nombre_producto,
        v_cantidad,
        v_precio_unit,
        v_total_linea
    IN
        SELECT
            lp.linea_id,
            lp.pedido_id,
            lp.producto_id,
            p.nombre,
            lp.cantidad,
            lp.precio_unit,
            lp.cantidad * lp.precio_unit AS total_linea
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_reporte_lineas_return_table();
```

5. RETURN TABLE + cursores múltiples

Enunciado

Devolver todos los pedidos con su total calculado, usando un cursor explícito.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_pedidos_return_table()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_cliente_id INTEGER,
    v_total NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_pedidos CURSOR FOR
        SELECT pedido_id, cliente_id
        FROM tienda.pedidos
        ORDER BY pedido_id;

BEGIN
    OPEN c_pedidos;

    LOOP
        FETCH c_pedidos INTO v_pedido_id, v_cliente_id;
        EXIT WHEN NOT FOUND;

        SELECT SUM(cantidad * precio_unit)
        INTO v_total
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id;

        RETURN NEXT;
    END LOOP;

    CLOSE c_pedidos;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_totales_pedidos_return_table();
```

¿Quieres el siguiente módulo?

Puedo preparar:

- **Cursores con manejo de errores**
- **Cursores para ETL y batch jobs**
- **Cursores con transacciones largas**
- **Cursores con RETURN NEXT + RECORD dinámico**

Dime cuál quieres y te lo construyo con ejemplos completos y tu formato de variables.

ahora el scrip anterior con return query y despues Cursores con RETURN NEXT + RECORD dinámico
Perfecto Jose, aquí tienes **dos módulos completos**:

1. **El script anterior reescrito usando** `RETURN QUERY` (en vez de cursores + `RETURN NEXT`)
2. **Cursores con** `RETURN NEXT` + `RECORD` **dinámico** (nivel avanzado)

Todo con tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc. Y con Guided Links para profundizar:

- **Return Query**
- **Record dinámico**
- **Cursores explícitos**
- **Cursores implícitos**

1. Reescritura del módulo anterior usando `RETURN QUERY`

Aquí transformamos todas las funciones que usaban cursores + `RETURN NEXT` en versiones optimizadas con `RETURN QUERY`.

Ejemplo 1 — Líneas de pedido con RETURN QUERY

Enunciado

Devolver todas las líneas de pedido sin cursores, usando `RETURN QUERY`.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_return_query()
```

```
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
    FROM tienda.lineas_pedido
    ORDER BY linea_id;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_lineas_return_query();
```

Ejemplo 2 — Productos activos con RETURN QUERY

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_activos_return_query()
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC,
    v_stock INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT producto_id, nombre, precio, stock
    FROM tienda.productos
    WHERE activo = TRUE
    ORDER BY producto_id;
END;
$$;
```

Ejemplo 3 — SQL dinámico con RETURN QUERY

Enunciado

Filtrar productos por stock mínimo usando SQL dinámico.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_stock_min_return_query(
    p_stock_min INTEGER
)
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC,
    v_stock INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
BEGIN
    v_sql := format(
        'SELECT producto_id, nombre, precio, stock
         FROM tienda.productos
         WHERE stock >= %s
         ORDER BY producto_id',
        p_stock_min
    );

    RETURN QUERY EXECUTE v_sql;
END;
$$;
```

Ejemplo 4 — Reporte de líneas con JOIN + RETURN QUERY

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_reporte_lineas_return_query()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_nombre_producto TEXT,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        lp.linea_id,
        lp.pedido_id,
        lp.producto_id,
        p.nombre,
        lp.cantidad,
        lp.precio_unit,
        lp.cantidad * lp.precio_unit AS total_linea
    FROM tienda.lineas_pedido lp
    JOIN tienda.productos p ON p.producto_id = lp.producto_id
    ORDER BY lp.linea_id;
```

```
END;
$$;
```

Ejemplo 5 — Totales de pedidos con RETURN QUERY

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_pedidos_return_query()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_cliente_id INTEGER,
    v_total NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.pedido_id,
        p.cliente_id,
        (
            SELECT SUM(lp.cantidad * lp.precio_unit)
            FROM tienda.lineas_pedido lp
            WHERE lp.pedido_id = p.pedido_id
        ) AS total
    FROM tienda.pedidos p
    ORDER BY p.pedido_id;
END;
$$;
```

2. Cursores con RETURN NEXT + RECORD dinámico

Este módulo es **avanzado**:

- Usamos un cursor explícito
- Leemos filas en un `RECORD` dinámico
- Extraemos columnas dinámicamente
- Devolvemos filas con `RETURN NEXT`

Ejemplo 6 — Cursor dinámico + RECORD dinámico + RETURN NEXT

Enunciado

Crear una función que recorra cualquier tabla que se le pase como parámetro y devuelva sus filas usando un `RECORD` dinámico.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_dinamico(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

Debes especificar la estructura del RECORD:

sql

```
SELECT *
FROM tienda.fn_cursor_record_dinamico('productos')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

Ejemplo 7 — RECORD dinámico + extracción de campos

Enunciado

Recorrer una tabla dinámica y extraer campos específicos del RECORD.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_record_dinamico_extract(
    p_tabla TEXT
)
RETURNS TABLE (
    v_campo1 TEXT,
    v_campo2 TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_ext';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        v_campo1 := v_rec.*::text;   -- convierte toda la fila en texto
        v_campo2 := v_rec.(2)::text; -- segundo campo dinámico

        RETURN NEXT;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Ejemplo 8 — RECORD dinámico + cursores + lógica condicional

Enunciado

Recorrer cualquier tabla y devolver solo filas que cumplan una condición dinámica.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_record_dinamico_condicion(
    p_tabla TEXT,
    p_campo TEXT,
    p_valor TEXT
```

```
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_cond';
    v_rec RECORD;
BEGIN
    v_sql := format(
        'SELECT * FROM tienda.%I WHERE %I::text = %L ORDER BY 1',
        p_tabla, p_campo, p_valor
    );

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_record_dinamico_condicion('productos', 'nombre', 'Teclado
mecánico')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

MEJORAS

Opcion 1: Mejora recomendada para tu función

Tu función funciona bien conceptualmente, pero en PostgreSQL es mucho más eficiente, limpia y legible usar **`RETURN QUERY EXECUTE`** en lugar de declarar un cursor y recorrerlo manualmente con un bucle `LOOP`.

Puedes simplificarla de la siguiente manera:

SQL

```
CREATE OR REPLACE FUNCTION tienda.fn_record_dinamico_condicion(
    p_tabla TEXT,
    p_campo TEXT,
    p_valor TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    RETURN QUERY EXECUTE format(
        'SELECT * FROM tienda.%I WHERE %I::text = %L ORDER BY 1',
        p_tabla, p_campo, p_valor
    );
END;
$$;
```

Opción 2: Cambiar la salida a JSON o JSONB (La más flexible)

Si en lugar de devolver `SETOF RECORD` devuelves un `SETOF JSON`, **no necesitas definir la estructura de columnas al llamar la función**, ya que cada fila se devuelve como un objeto JSON dinámico.

Aquí tienes cómo quedaría adaptada:

SQL

```
CREATE OR REPLACE FUNCTION tienda.fn_json_dinamico_condicion(
    p_tabla TEXT,
    p_campo TEXT,
    p_valor TEXT
)
RETURNS SETOF JSON
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
BEGIN
    -- Quitamos el "ORDER BY 1"
    v_sql := format(
        'SELECT row_to_json(t) FROM tienda.%I AS t WHERE t.%I::text = %L',
        p_tabla, p_campo, p_valor
    );

    RETURN QUERY EXECUTE v_sql;
END;
$$;
```

¿Cómo se llama esta versión con JSON?

¡Mucho más limpia y sin necesidad de escribir el esquema de columnas!

SQL

```
SELECT tienda.fn_json_dinamico_condicion('clientes', 'ciudad', 'Madrid');
```

Esto te devolverá directamente un resultado en formato JSON por cada fila (por ejemplo: `{"id_cliente": 1, "nombre": "Ana", "ciudad": "Madrid"}`), lo cual es ideal si estás consumiendo esta base de datos desde una API o una aplicación web.

'SELECT * FROM tienda.%I WHERE %I::text = %L ORDER BY 1',

p_tabla, p_campo, p_valor **porque se ponen los tres valores al final, p_tabla,p_campo,p_valor ?**

Esos tres valores se colocan al final porque `format()` funciona como una plantilla de texto, donde cada símbolo especial (`%I` o `%L`) busca su valor correspondiente en el orden exacto en que los vas listando.

Para entenderlo mejor, veamos cómo se relacionan uno a uno:

Posición del marcador	Marcador en la cadena	Variable que lo reemplaza	Qué tipo de dato maneja
1º	`%I` (primer `%I`)	`p_tabla`	Identificador (nombre de la tabla)
2º	`%I` (segundo `%I`)	`p_campo`	Identificador (nombre de la columna)
3º	`%L`	`p_valor`	Literal (el valor a buscar, ej: texto o número, escapado de forma segura)

¿Cómo lo procesa PostgreSQL?

Cuando se ejecuta esta línea:

SQL

```
v_sql := format(
    'SELECT * FROM tienda.%I WHERE %I::text = %L ORDER BY 1',
    p_tabla, p_campo, p_valor
);
```

PostgreSQL toma la cadena de texto y va reemplazando secuencialmente:

1. El primer `%I` se sustituye por el valor del primer argumento (`p_tabla`).
2. El segundo `%I` se sustituye por el valor del segundo argumento (`p_campo`).
3. El `%L` se sustituye de forma segura por el valor del tercer argumento (`p_valor`), colocándole comillas automáticamente para evitar errores de sintaxis o ataques de inyección SQL (SQL Injection).

   Por ejemplo, si los parámetros son `'productos'`, `'nombre'` y `'Teclado mecánico'`, el resultado final de `v_sql` será exactamente:

   SQL

   ```
   SELECT * FROM tienda.productos WHERE nombre::text = 'Teclado mecánico' ORDER BY
   1
   ```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores con manejo de errores**
- **Cursores para ETL y batch jobs**
- **Cursores con transacciones largas**
- **Cursores con estructuras complejas**

Dime cuál quieres y te lo construyo con ejemplos completos y tu formato de variables.

y esto como lo hariamos con return query:LOOP FETCH c_pedidos INTO v_pedido_id, v_cliente_id; EXIT WHEN NOT FOUND; SELECT SUM(cantidad * precio_unit) INTO v_total FROM tienda.lineas_pedido WHERE pedido_id = v_pedido_id; RETURN NEXT; END LOOP;
Aquí tienes, Jose — **la versión EXACTA de ese bloque usando** `RETURN QUERY`, sin cursores, sin `FETCH`, sin `RETURN NEXT`, y totalmente optimizada.

Además, al final te dejo **el módulo avanzado de Cursores** + `RETURN NEXT` + **RECORD dinámico**, como pediste.

Incluyo Guided Links para profundizar:

- **Return Query**
- **Cursores dinámicos**
- **Record dinámico**

1. Cómo convertir tu bloque a `RETURN QUERY`

Tu bloque original:

plpgsql

```
LOOP
    FETCH c_pedidos INTO v_pedido_id, v_cliente_id;
    EXIT WHEN NOT FOUND;

    SELECT SUM(cantidad * precio_unit)
    INTO v_total
    FROM tienda.lineas_pedido
    WHERE pedido_id = v_pedido_id;

    RETURN NEXT;
END LOOP;
```

Este patrón es típico:

1. Recorrer pedidos

2. Calcular el total
3. Devolver fila por fila

Con `RETURN QUERY`, **todo se reemplaza por una sola sentencia SQL**.

Versión optimizada con `RETURN QUERY`

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_pedidos_return_query()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_cliente_id INTEGER,
    v_total NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.pedido_id,
        p.cliente_id,
        (
            SELECT SUM(lp.cantidad * lp.precio_unit)
            FROM tienda.lineas_pedido lp
            WHERE lp.pedido_id = p.pedido_id
        ) AS v_total
    FROM tienda.pedidos p
    ORDER BY p.pedido_id;
END;
$$;
```

Resultado

sql

```
SELECT * FROM tienda.fn_totales_pedidos_return_query();
```

¿Por qué esta versión es mejor?

- No usa cursores
- No usa FETCH
- No usa RETURN NEXT
- No necesita variables internas
- Es más rápida
- Es más legible
- PostgreSQL puede optimizar el plan de ejecución

2. Módulo avanzado: Cursores + RETURN NEXT + RECORD dinámico

Aquí tienes el módulo completo que pediste, con ejemplos avanzados.

Ejemplo 1 — Cursor dinámico + RECORD dinámico + RETURN NEXT

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_dinamico(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_record_dinamico('productos')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

Ejemplo 2 — RECORD dinámico + extracción de campos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_record_dinamico_extract(
    p_tabla TEXT
)
RETURNS TABLE (
    v_campo1 TEXT,
    v_campo2 TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_ext';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        v_campo1 := v_rec.*::text;
        v_campo2 := v_rec.(2)::text;

        RETURN NEXT;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Ejemplo 3 — RECORD dinámico + condición dinámica

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_record_dinamico_condicion(
    p_tabla TEXT,
    p_campo TEXT,
    p_valor TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_cond';
    v_rec RECORD;
BEGIN
    v_sql := format(
        'SELECT * FROM tienda.%I WHERE %I::text = %L ORDER BY 1',
        p_tabla, p_campo, p_valor
    );

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
```

```
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_record_dinamico_condicion('productos', 'nombre', 'Teclado
mecánico')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores con manejo de errores**
- **Cursores para ETL y batch jobs**
- **Cursores con transacciones largas**
- **Cursores con estructuras complejas**

Dime cuál quieres y te lo dejo perfecto con tu formato de variables.

cursores manejo errores postgresql
Aquí tienes, Jose — **el módulo completo de “Cursores con manejo de errores en PostgreSQL 18”**, con ejemplos avanzados, claros, y usando SIEMPRE tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc.

Incluyo Guided Links para profundizar:

- **Cursores manejo errores**
- **Raise Exception**
- **Bloques EXCEPTION**
- **Cursores explícitos**

1. Conceptos clave del manejo de errores con cursores

En PL/pgSQL puedes capturar errores con:

plpgsql

```
BEGIN
    ...
EXCEPTION
    WHEN others THEN
        ...
END;
```

Esto permite:

- Detectar errores al abrir cursores
- Detectar errores al hacer FETCH
- Detectar errores en SQL dinámico
- Registrar errores sin detener la ejecución
- Reintentar operaciones

2. Ejemplo básico — Manejo de errores al abrir un cursor

Enunciado

Intentar abrir un cursor sobre una tabla que podría no existir.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_error_open(
    p_tabla TEXT
)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_err';
BEGIN
    BEGIN
        OPEN v_cursor FOR EXECUTE format('SELECT * FROM tienda.%I', p_tabla);
    EXCEPTION
        WHEN undefined_table THEN
            RETURN 'Error: la tabla ' || p_tabla || ' no existe';
        WHEN others THEN
            RETURN 'Error inesperado al abrir cursor';
    END;

    RETURN 'Cursor abierto correctamente';
END;
$$;
```

Uso

sql

```
SELECT tienda.fn_cursor_error_open('productos');
SELECT tienda.fn_cursor_error_open('tabla_inexistente');
```

3. Manejo de errores en FETCH

Enunciado

Capturar errores cuando una fila no puede ser leída.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_error_fetch()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_fetch';
    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN v_cursor FOR
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    LOOP
        BEGIN
            FETCH v_cursor INTO v_linea_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE 'Línea %, cantidad %', v_linea_id, v_cantidad;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error al hacer FETCH: %', SQLERRM;
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

4. Manejo de errores en SQL dinámico dentro de cursores

Enunciado

Intentar ejecutar SQL dinámico que puede fallar.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_sql_dinamico_error(
    p_sql TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    v_cursor refcursor := 'cur_dyn_err';
    v_rec RECORD;
BEGIN
    BEGIN
        OPEN v_cursor FOR EXECUTE p_sql;
    EXCEPTION
        WHEN syntax_error THEN
            RAISE EXCEPTION 'Error de sintaxis en SQL dinámico: %', p_sql;
        WHEN others THEN
            RAISE EXCEPTION 'Error inesperado: %', SQLERRM;
    END;

    LOOP
        BEGIN
            FETCH v_cursor INTO v_rec;
            EXIT WHEN NOT FOUND;

            RETURN NEXT v_rec;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error al leer fila: %', SQLERRM;
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_sql_dinamico_error('SELECT * FROM tienda.productos')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

5. Manejo de errores + cursores + RETURN TABLE

Enunciado

Devolver totales de pedidos, pero capturar errores si una línea tiene datos corruptos.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_pedidos_error_return_table()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_cliente_id INTEGER,
    v_total NUMERIC,
    v_error TEXT
)
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    c_pedidos CURSOR FOR
        SELECT pedido_id, cliente_id FROM tienda.pedidos ORDER BY pedido_id;

BEGIN
    OPEN c_pedidos;

    LOOP
        FETCH c_pedidos INTO v_pedido_id, v_cliente_id;
        EXIT WHEN NOT FOUND;

        BEGIN
            SELECT SUM(cantidad * precio_unit)
            INTO v_total
            FROM tienda.lineas_pedido
            WHERE pedido_id = v_pedido_id;

            v_error := NULL;

        EXCEPTION
            WHEN others THEN
                v_total := NULL;
                v_error := SQLERRM;
        END;

        RETURN NEXT;
    END LOOP;

    CLOSE c_pedidos;
END;
$$;
```

6. Manejo de errores + cursores + reintentos

Enunciado

Reintentar un FETCH hasta 3 veces si falla.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_reintentos()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_retry';
    v_linea_id INTEGER;
    v_cantidad INTEGER;
    v_intentos INTEGER;
BEGIN
    OPEN v_cursor FOR
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    LOOP
```

```
        v_intentos := 0;

        <<reintentar>>
        BEGIN
            FETCH v_cursor INTO v_linea_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE 'Línea %, cantidad %', v_linea_id, v_cantidad;

        EXCEPTION
            WHEN others THEN
                v_intentos := v_intentos + 1;

                IF v_intentos < 3 THEN
                    RAISE NOTICE 'Error en FETCH, reintentando (%).',
v_intentos;
                    GOTO reintentar;
                ELSE
                    RAISE NOTICE 'Error definitivo: %', SQLERRM;
                END IF;
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

7. Manejo de errores + cursores + logs en tabla

Enunciado

Registrar errores en una tabla de logs.

sql

```
CREATE TABLE IF NOT EXISTS tienda.logs_errores (
    id SERIAL PRIMARY KEY,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    mensaje TEXT
);
```

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_log_errores()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_log';
    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN v_cursor FOR
```

```
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    LOOP
        BEGIN
            FETCH v_cursor INTO v_linea_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE 'Línea %, cantidad %', v_linea_id, v_cantidad;

        EXCEPTION
            WHEN others THEN
                INSERT INTO tienda.logs_errores(mensaje)
                VALUES ('Error en FETCH: ' || SQLERRM);
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores para ETL y batch jobs**
- **Cursores con transacciones largas**
- **Cursores con estructuras complejas**
- **Cursores con SAVEPOINT**

Dime cuál quieres y te lo dejo perfecto con tu formato de variables.

cursores transacciones largas postgresql
Jose, aquí tienes **EL MÓDULO DEFINITIVO DE CURSORES CON TRANSACCIONES LARGAS EN POSTGRESQL 18**, totalmente adaptado a tu estilo, con ejemplos reales, robustos, y usando SIEMPRE tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc.

Incluyo Guided Links para profundizar:

- **Transacciones largas**
- **Cursores explícitos**
- **SAVEPOINT**
- **Bloques EXCEPTION**

1. ¿Qué son las transacciones largas con cursores?

Una **transacción larga** es aquella que:

- Procesa miles o millones de filas
- Mantiene un cursor abierto durante mucho tiempo
- Puede fallar en mitad del proceso
- Necesita **SAVEPOINT**, **EXCEPTION**, **reintentos**, **commits parciales**, etc.

En PostgreSQL, los cursores dentro de transacciones largas se usan para:

- ETL
- Migraciones
- Procesamiento batch
- Limpieza de datos
- Recalcular totales
- Reindexar información

2. Ejemplo 1 — Cursor + transacción larga + SAVEPOINT

Enunciado

Recorrer todas las líneas de pedido, recalcular su total y registrar errores sin detener la transacción.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_transaccion_larga_lineas()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_lineas';
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;
    v_total NUMERIC;
BEGIN
    -- Abrimos cursor
    OPEN v_cursor FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;

    LOOP
        FETCH v_cursor INTO
            v_linea_id,
            v_pedido_id,
```

```
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        -- SAVEPOINT para evitar abortar toda la transacción
        SAVEPOINT sp_linea;

        BEGIN
            v_total := v_cantidad * v_precio_unit;

            RAISE NOTICE 'Procesando línea %, total %', v_linea_id, v_total;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error en línea %: %', v_linea_id, SQLERRM;
                ROLLBACK TO SAVEPOINT sp_linea;
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

3. Ejemplo 2 — Transacción larga con COMMIT parciales

Enunciado

Procesar miles de productos y hacer **COMMIT cada 1000 filas** para evitar:

- Bloqueos prolongados
- Crecimiento del WAL
- Retención de MVCC

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_transaccion_larga_commit_parcial()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_prod';
    v_producto_id INTEGER;
    v_nombre TEXT;
    v_precio NUMERIC;
    v_stock INTEGER;
    v_contador INTEGER := 0;
BEGIN
    OPEN v_cursor FOR
        SELECT producto_id, nombre, precio, stock
```

```
        FROM tienda.productos
        ORDER BY producto_id;

    LOOP
        FETCH v_cursor INTO
            v_producto_id,
            v_nombre,
            v_precio,
            v_stock;

        EXIT WHEN NOT FOUND;

        v_contador := v_contador + 1;

        RAISE NOTICE 'Procesando producto %', v_producto_id;

        IF v_contador % 1000 = 0 THEN
            RAISE NOTICE 'Commit parcial en fila %', v_contador;
            COMMIT;
            START TRANSACTION;
        END IF;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

4. Ejemplo 3 — Transacción larga + cursor + reintentos automáticos

Enunciado

Si una fila falla, reintentar hasta 3 veces antes de registrarla como error.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_transaccion_larga_reintentos()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_retry';
    v_linea_id INTEGER;
    v_cantidad INTEGER;
    v_intentos INTEGER;
BEGIN
    OPEN v_cursor FOR
        SELECT linea_id, cantidad
        FROM tienda.lineas_pedido
        ORDER BY linea_id;

    LOOP
        FETCH v_cursor INTO v_linea_id, v_cantidad;
        EXIT WHEN NOT FOUND;
```

```
        v_intentos := 0;

        <<reintentar>>
        BEGIN
            RAISE NOTICE 'Procesando línea %', v_linea_id;

            -- Simulación de error
            IF v_linea_id = 2 THEN
                RAISE EXCEPTION 'Error simulado en línea 2';
            END IF;

        EXCEPTION
            WHEN others THEN
                v_intentos := v_intentos + 1;

                IF v_intentos < 3 THEN
                    RAISE NOTICE 'Reintentando línea % (intento %)', v_linea_id,
v_intentos;
                    GOTO reintentar;
                ELSE
                    RAISE NOTICE 'Error definitivo en línea %: %', v_linea_id,
SQLERRM;
                END IF;
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

5. Ejemplo 4 — Transacción larga + cursor + tabla de logs

Enunciado

Registrar errores en una tabla sin abortar la transacción.

Tabla de logs

sql

```
CREATE TABLE IF NOT EXISTS tienda.logs_transacciones (
    id SERIAL PRIMARY KEY,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    v_linea_id INTEGER,
    mensaje TEXT
);
```

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_transaccion_larga_log()
```

```
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_log';
    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN v_cursor FOR
        SELECT linea_id, cantidad
        FROM tienda.lineas_pedido
        ORDER BY linea_id;

    LOOP
        FETCH v_cursor INTO v_linea_id, v_cantidad;
        EXIT WHEN NOT FOUND;

        BEGIN
            RAISE NOTICE 'Procesando línea %', v_linea_id;

            -- Simulación de error
            IF v_linea_id = 3 THEN
                RAISE EXCEPTION 'Error simulado en línea 3';
            END IF;

        EXCEPTION
            WHEN others THEN
                INSERT INTO tienda.logs_transacciones(v_linea_id, mensaje)
                VALUES (v_linea_id, SQLERRM);
        END;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

6. Ejemplo 5 — Transacción larga + cursor + RETURN TABLE

Enunciado

Procesar pedidos y devolver:

- Totales
- Estado
- Errores capturados

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_transaccion_larga_return_table()
RETURNS TABLE (
    v_pedido_id INTEGER,
```

```
    v_total NUMERIC,
    v_estado TEXT,
    v_error TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor := 'cur_ped';
    v_pedido_id INTEGER;
    v_total NUMERIC;
BEGIN
    OPEN v_cursor FOR
        SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;

    LOOP
        FETCH v_cursor INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        BEGIN
            SELECT SUM(cantidad * precio_unit)
            INTO v_total
            FROM tienda.lineas_pedido
            WHERE pedido_id = v_pedido_id;

            v_estado := 'OK';
            v_error := NULL;

        EXCEPTION
            WHEN others THEN
                v_total := NULL;
                v_estado := 'ERROR';
                v_error := SQLERRM;
        END;

        RETURN NEXT;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores ETL y batch jobs**
- **Cursores con estructuras complejas**
- **Cursores con SAVEPOINT avanzado**
- **Cursores para auditoría y replicación**

Dime cuál quieres y te lo dejo perfecto con tu formato de variables.

cursores estructuras complejas postgresql
Perfecto, Jose. Vamos con **CURSORES + ESTRUCTURAS COMPLEJAS en PostgreSQL 18**, el módulo más avanzado del curso. Aquí verás cursores trabajando con:

- **Registros compuestos (ROW TYPE)**
- **Tipos personalizados (CREATE TYPE)**
- **JSON / JSONB**
- **Arrays**
- **Tablas temporales**
- **Cursores múltiples sincronizados**
- **Cursores con estructuras jerárquicas**
- **Cursores con RETURN TABLE y RECORD dinámico**

Todo con tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc.

Incluyo Guided Links para profundizar:

- **Cursores estructuras complejas**
- **Tipos compuestos**
- **JSON en PostgreSQL**
- **Arrays PostgreSQL**

1. Cursores con tipos compuestos (ROW TYPE)

Enunciado

Recorrer líneas de pedido usando un **tipo compuesto** basado en la tabla.

Ejemplo 1 — Cursor con `%ROWTYPE`

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    c_lineas CURSOR FOR
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;
```

```
        RAISE NOTICE 'Línea %, Pedido %, Producto %, Cantidad %',
            v_registro.linea_id,
            v_registro.pedido_id,
            v_registro.producto_id,
            v_registro.cantidad;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

2. Cursores con tipos personalizados (CREATE TYPE)

Enunciado

Crear un tipo complejo para representar un pedido completo.

Tipo personalizado

sql

```
CREATE TYPE tienda.tipo_pedido_completo AS (
    v_pedido_id INTEGER,
    v_cliente_id INTEGER,
    v_total NUMERIC,
    v_num_lineas INTEGER
);
```

Función con cursor + tipo compuesto

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_tipo_complejo()
RETURNS SETOF tienda.tipo_pedido_completo
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg tienda.tipo_pedido_completo;
    c_ped CURSOR FOR
        SELECT pedido_id, cliente_id FROM tienda.pedidos ORDER BY pedido_id;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_reg.v_pedido_id, v_reg.v_cliente_id;
        EXIT WHEN NOT FOUND;

        SELECT SUM(cantidad * precio_unit),
               COUNT(*)
        INTO v_reg.v_total, v_reg.v_num_lineas
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_reg.v_pedido_id;
```

```
        RETURN NEXT v_reg;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

3. Cursores con JSON / JSONB

Enunciado

Generar un JSON por cada pedido usando cursores.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_json()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_json JSONB
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        v_json := (
            SELECT jsonb_agg(
                jsonb_build_object(
                    'linea_id', linea_id,
                    'producto_id', producto_id,
                    'cantidad', cantidad,
                    'precio_unit', precio_unit
                )
            )
            FROM tienda.lineas_pedido
            WHERE pedido_id = v_pedido_id
        );

        RETURN NEXT;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

4. Cursores con arrays

Enunciado

Crear un array con los IDs de productos de cada pedido.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_arrays()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_productos INTEGER[]
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        SELECT array_agg(producto_id)
        INTO v_productos
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id;

        RETURN NEXT;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

5. Cursores múltiples sincronizados (JOIN manual)

Enunciado

Recorrer pedidos y líneas simultáneamente con dos cursores.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_doble()
RETURNS VOID
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    c_lin CURSOR (p_id INTEGER) FOR
        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_id;

    v_pedido_id INTEGER;
    v_linea_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Pedido %:', v_pedido_id;

        OPEN c_lin(v_pedido_id);

        LOOP
            FETCH c_lin INTO v_linea_id, v_producto_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE '  Línea %, Producto %, Cantidad %',
                v_linea_id, v_producto_id, v_cantidad;
        END LOOP;

        CLOSE c_lin;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

6. Cursores con estructuras jerárquicas (árboles)

Enunciado

Simular una jerarquía:

- Departamento → Empleados

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_jerarquia()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c_dep CURSOR FOR
```

```
        SELECT DISTINCT departamento FROM tienda.empleados ORDER BY
departamento;

    c_emp CURSOR (p_dep TEXT) FOR
        SELECT nombre, salario FROM tienda.empleados WHERE departamento = p_dep;

    v_dep TEXT;
    v_nombre TEXT;
    v_salario NUMERIC;
BEGIN
    OPEN c_dep;

    LOOP
        FETCH c_dep INTO v_dep;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Departamento: %', v_dep;

        OPEN c_emp(v_dep);

        LOOP
            FETCH c_emp INTO v_nombre, v_salario;
            EXIT WHEN NOT FOUND;

            RAISE NOTICE '  Empleado: %, Salario %', v_nombre, v_salario;
        END LOOP;

        CLOSE c_emp;
    END LOOP;

    CLOSE c_dep;
END;
$$;
```

7. Cursores + estructuras complejas + RETURN TABLE

Enunciado

Devolver un reporte completo:

- Pedido
- Total
- JSON de líneas
- Array de productos
- Número de líneas

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_estructura_compleja()
```

```
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_total NUMERIC,
    v_json_lineas JSONB,
    v_productos INTEGER[],
    v_num_lineas INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        SELECT SUM(cantidad * precio_unit),
               jsonb_agg(jsonb_build_object(
                   'linea_id', linea_id,
                   'producto_id', producto_id,
                   'cantidad', cantidad,
                   'precio_unit', precio_unit
               )),
               array_agg(producto_id),
               COUNT(*)
        INTO v_total, v_json_lineas, v_productos, v_num_lineas
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id;

        RETURN NEXT;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores ETL y batch jobs**
- **Cursores con SAVEPOINT avanzado**
- **Cursores para auditoría y replicación**
- **Cursores con paralelismo simulado**

Dime cuál quieres y te lo dejo perfecto con tu formato de variables.

esto : v_registro tienda.lineas_pedido%ROWTYPE; equivale a esto v_registro setof tienda.lineas_pedido ?
En absoluto, Jose — **no son equivalentes**. Y es una duda muy común cuando uno empieza a trabajar con cursores y estructuras complejas en PostgreSQL.

Vamos directo al grano, con precisión técnica y ejemplos claros.

1. ¿`v_registro tienda.lineas_pedido%ROWTYPE` equivale a `v_registro SETOF tienda.lineas_pedido`?

No. Son conceptos totalmente distintos.

`%ROWTYPE` → una sola fila (un registro)

`v_registro tienda.lineas_pedido%ROWTYPE` significa:

> "v_registro es una variable que contiene **una sola fila** con la estructura de la tabla `tienda.lineas_pedido`."

Es decir, es un **registro individual**, con campos:

- `v_registro.linea_id`
- `v_registro.pedido_id`
- `v_registro.producto_id`
- `v_registro.cantidad`
- `v_registro.precio_unit`

Es **una fila**, no un conjunto.

`SETOF tienda.lineas_pedido` → muchas filas (un conjunto)

`SETOF tienda.lineas_pedido` significa:

> "La función devuelve **muchas filas**, cada una con la estructura de la tabla."

Esto no es una variable. Es un **tipo de retorno de función**.

Ejemplo:

sql

```
RETURNS SETOF tienda.lineas_pedido
```

Esto indica que la función devolverá **varias filas**, no una sola.

2. Comparación directa

Concepto	%ROWTYPE	SETOF
Representa	Una fila	Muchas filas
Se usa en	Variables	Firma de funciones
Contiene	Campos individuales	Conjunto de registros
Ejemplo	`v_registro tienda.lineas_pedido%ROWTYPE`	`RETURNS SETOF tienda.lineas_pedido`

3. Ejemplo práctico para verlo claro

Ejemplo A — %ROWTYPE dentro de un cursor

sql

```
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    c_lineas CURSOR FOR SELECT * FROM tienda.lineas_pedido;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Cantidad %',
            v_registro.linea_id,
            v_registro.cantidad;
    END LOOP;

    CLOSE c_lineas;
END;
```

Aquí `v_registro` es **una fila cada vez**.

Ejemplo B — SETOF tienda.lineas_pedido en una función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas()
RETURNS SETOF tienda.lineas_pedido
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT * FROM tienda.lineas_pedido;
END;
$$;
```

Aquí la función devuelve **todas las filas** de la tabla.

4. ¿Cómo sería una variable que contenga “muchas filas”?

PostgreSQL **no permite** declarar una variable como:

sql

```
v_registro SETOF tienda.lineas_pedido;
```

Eso **no existe**.

Si quieres almacenar varias filas en una variable, debes usar:

Arrays

sql

```
v_registros tienda.lineas_pedido[];
```

JSONB

sql

```
v_registros JSONB;
```

Tablas temporales

sql

```
CREATE TEMP TABLE tmp AS SELECT ...
```

Cursores

sql

```
v_cursor refcursor;
```

5. Ejemplo avanzado: almacenar varias filas en una variable JSONB

sql

```
DECLARE
    v_registros JSONB;
BEGIN
    SELECT jsonb_agg(to_jsonb(lp))
    INTO v_registros
    FROM tienda.lineas_pedido lp;

    RAISE NOTICE '%', v_registros;
END;
```

6. Ejemplo avanzado: almacenar varias filas en un array de registros

PostgreSQL no soporta arrays de tipos compuestos sin definir un tipo, así que:

1. Definimos un tipo:

sql

```
CREATE TYPE tienda.tipo_linea AS (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

2. Creamos un array de ese tipo:

sql

```
DECLARE
    v_registros tienda.tipo_linea[];
BEGIN
    SELECT array_agg(
        ROW(linea_id, pedido_id, producto_id, cantidad,
precio_unit)::tienda.tipo_linea
    )
    INTO v_registros
    FROM tienda.lineas_pedido;

    RAISE NOTICE '%', v_registros;
END;
```

Resumen final

No equivalen

`v_registro tienda.lineas_pedido%ROWTYPE` ≠ `v_registro SETOF tienda.lineas_pedido`

`%ROWTYPE` = una fila

`SETOF` = muchas filas (solo en RETURNS)

¿Quieres que prepare ahora…?

- **Cursores ETL y batch jobs**
- **Cursores con SAVEPOINT avanzado**

- **Cursores con paralelismo simulado**
- **Cursores para auditoría y replicación**

Dime cuál módulo quieres y te lo dejo perfecto con tu formato de variables.

y esto: v_registro tienda.lineas_pedido%ROWTYPE; equivale a esto setof ienda.lineas_pedido %ROWTYPE ?
Jose, vamos a dejarlo **cristalino** porque esta duda es clave cuando trabajas con cursores y estructuras complejas en PostgreSQL:

Respuesta corta y directa

NO. `v_registro tienda.lineas_pedido%ROWTYPE` **NO equivale** a `SETOF tienda.lineas_pedido%ROWTYPE`.

Tampoco equivale a: `v_registro SETOF tienda.lineas_pedido%ROWTYPE`.

Son conceptos **totalmente diferentes**.

1. `%ROWTYPE` = una sola fila

Cuando declaras:

plpgsql

```
v_registro tienda.lineas_pedido%ROWTYPE;
```

Estás diciendo:

> "v_registro es una variable que contiene **una sola fila**, con la estructura de la tabla `tienda.lineas_pedido`."

Es decir, v_registro tiene estos campos:

- `v_registro.linea_id`
- `v_registro.pedido_id`
- `v_registro.producto_id`
- `v_registro.cantidad`
- `v_registro.precio_unit`

Es un registro individual.

2. `SETOF` = muchas filas (solo en RETURNS)

Cuando escribes:

plpgsql

```
RETURNS SETOF tienda.lineas_pedido
```

Significa:

> “La función devolverá **muchas filas**, cada una con la estructura de la tabla.”

SETOF solo se usa en la firma de una función. No se puede usar para declarar variables.

3. Esto NO existe en PostgreSQL

No puedes declarar:

plpgsql

```
v_registro SETOF tienda.lineas_pedido%ROWTYPE;
```

Ni:

plpgsql

```
v_registro SETOF tienda.lineas_pedido;
```

Ni:

plpgsql

```
v_registro SETOF RECORD;
```

PostgreSQL no permite variables que contengan “muchas filas” directamente.

4. Comparación clara

Concepto	%ROWTYPE	SETOF
Representa	Una fila	Muchas filas
Se usa en	Variables	Firma de funciones
Contiene	Campos individuales	Conjunto de registros
Ejemplo válido	`v_registro tienda.lineas_pedido%ROWTYPE`	`RETURNS SETOF tienda.lineas_pedido`
Ejemplo inválido	`v_registro SETOF tienda.lineas_pedido`	—

5. Ejemplo real con %ROWTYPE (válido)

plpgsql

```
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    c_lineas CURSOR FOR SELECT * FROM tienda.lineas_pedido;
```

```
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Cantidad %',
            v_registro.linea_id,
            v_registro.cantidad;
    END LOOP;

    CLOSE c_lineas;
END;
```

Aquí `v_registro` es **una fila cada vez**.

6. Ejemplo real con `SETOF` (válido)

plpgsql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas()
RETURNS SETOF tienda.lineas_pedido
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT * FROM tienda.lineas_pedido;
END;
$$;
```

Aquí la función devuelve **muchas filas**.

7. ¿Cómo almacenar varias filas en una variable? (lo que buscabas)

PostgreSQL **no permite** variables tipo SETOF, pero sí puedes almacenar múltiples filas usando:

JSONB

plpgsql

```
DECLARE
    v_registros JSONB;
BEGIN
    SELECT jsonb_agg(to_jsonb(lp))
    INTO v_registros
    FROM tienda.lineas_pedido lp;
END;
```

Arrays de tipos compuestos

plpgsql

```
CREATE TYPE tienda.tipo_linea AS (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);

DECLARE
    v_registros tienda.tipo_linea[];
BEGIN
    SELECT array_agg(
        ROW(linea_id, pedido_id, producto_id, cantidad,
precio_unit)::tienda.tipo_linea
    )
    INTO v_registros
    FROM tienda.lineas_pedido;
END;
```

Tablas temporales

sql

```
CREATE TEMP TABLE tmp AS SELECT * FROM tienda.lineas_pedido;
```

Cursores

plpgsql

```
DECLARE
    v_cursor refcursor;
```

Resumen final

No equivalen

`v_registro tienda.lineas_pedido%ROWTYPE` ≠ `SETOF tienda.lineas_pedido %ROWTYPE`

`%ROWTYPE` = una fila

`SETOF` = muchas filas (solo en RETURNS)

Para “muchas filas en variable”: JSONB, arrays, cursores, tablas temporales

Si quieres, puedo prepararte:

- **Cómo simular SETOF dentro de variables**
- **Cursores avanzados con tipos compuestos**
- **Cursores + JSONB + estructuras complejas**

¿Quieres que te construya uno de esos módulos?

y esto se puede hacer return setof tienda.lineas_pedido%ROWTYPE?
Jose, vamos a dejar esto **cerrado al 100%** porque es una duda muy técnica y muy importante cuando trabajas con PL/pgSQL:

Respuesta directa

NO. No se puede hacer:

sql

```
RETURN SETOF tienda.lineas_pedido%ROWTYPE;
```

Y tampoco:

sql

```
RETURNS SETOF tienda.lineas_pedido%ROWTYPE;
```

PostgreSQL **no permite** usar `%ROWTYPE` dentro de `SETOF`.

¿Por qué NO se puede?

Porque:

`%ROWTYPE` es solo para variables

Sirve para declarar **una sola fila** con la estructura de una tabla:

plpgsql

```
v_registro tienda.lineas_pedido%ROWTYPE;
```

`%ROWTYPE` NO es un tipo de retorno válido

PostgreSQL **no permite** usar `%ROWTYPE` en la firma de una función.

¿Qué sí se puede usar con SETOF?

SETOF RECORD

sql

```
RETURNS SETOF RECORD
```

SETOF <tipo compuesto>

sql

```
RETURNS SETOF tienda.tipo_linea
```

SETOF <tabla>

sql

```
RETURNS SETOF tienda.lineas_pedido
```

Comparación clara

Concepto	¿Se puede usar en variables?	¿Se puede usar en RETURNS?
%ROWTYPE	Sí	No
SETOF RECORD	No	Sí
SETOF <tabla>	No	Sí
SETOF <tipo compuesto>	No	Sí

Ejemplo válido con %ROWTYPE (variable)

plpgsql

```
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
BEGIN
    SELECT * INTO v_registro
    FROM tienda.lineas_pedido
    WHERE linea_id = 1;
END;
```

Ejemplo válido con RETURNS SETOF tienda.lineas_pedido

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas()
RETURNS SETOF tienda.lineas_pedido
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT * FROM tienda.lineas_pedido;
END;
$$;
```

Ejemplo válido con RETURNS SETOF RECORD

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_record()
RETURNS SETOF RECORD
```

```
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT linea_id, cantidad
    FROM tienda.lineas_pedido;
END;
$$;
```

Uso:

sql

```
SELECT *
FROM tienda.fn_lineas_record() AS t(
    v_linea_id INTEGER,
    v_cantidad INTEGER
);
```

Ejemplo válido con tipo compuesto

1. Crear tipo

sql

```
CREATE TYPE tienda.tipo_linea AS (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

2. Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_tipo()
RETURNS SETOF tienda.tipo_linea
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
    FROM tienda.lineas_pedido;
END;
$$;
```

Resumen final

No se puede

```
RETURN SETOF tienda.lineas_pedido%ROWTYPE
```

Sí se puede

- `RETURN SETOF tienda.lineas_pedido`
- `RETURN SETOF RECORD`
- `RETURN SETOF tienda.tipo_linea`

`%ROWTYPE` solo sirve para variables

Nunca para tipos de retorno.

Si quieres, puedo prepararte:

- **Cómo convertir cualquier %ROWTYPE en un tipo compuesto**
- **Funciones avanzadas SETOF RECORD con cursores**
- **Diseño profesional de tipos compuestos para APIs SQL**

Dime cuál módulo quieres y te lo dejo perfecto con tu formato de variables.

funciones setof record cursores postgresql
Perfecto, Jose. Vamos a construir **EL MÓDULO DEFINITIVO DE FUNCIONES `SETOF RECORD` con CURSORES en PostgreSQL 18**, con ejemplos avanzados, claros, y usando SIEMPRE tu formato de variables: `v_linea_id`, `v_pedido_id`, `v_producto_id`, `v_cantidad`, `v_precio_unit`, etc.

Incluyo Guided Links para profundizar:

- **SETOF RECORD**
- **Cursores explícitos**
- **Cursores dinámicos**
- **Record dinámico**

1. Concepto clave: ¿Qué es `SETOF RECORD`?

`SETOF RECORD` permite devolver **filas con estructura dinámica**, es decir, la función no define las columnas de salida; el usuario las define al llamar la función.

Esto es ideal cuando:

- La estructura depende de SQL dinámico
- La estructura depende de parámetros
- Se usan cursores que devuelven distintas columnas
- Se procesan tablas variables

2. Ejemplo base — Cursor explícito + `SETOF RECORD`

Enunciado

Recorrer líneas de pedido con un cursor explícito y devolverlas como `SETOF RECORD`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;

    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT ROW(
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit
        );
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Uso (definiendo estructura)

sql

```
SELECT *
FROM tienda.fn_lineas_setof_record() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

3. Cursor dinámico + SETOF RECORD

Enunciado

Devolver cualquier tabla de la base de datos usando SQL dinámico y cursores.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_tabla_setof_record(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_tabla_setof_record('productos')
    AS t(
        v_producto_id INTEGER,
        v_nombre TEXT,
        v_precio NUMERIC,
        v_stock INTEGER,
```

```
        v_activo BOOLEAN
    );
```

4. SETOF RECORD + cursores + JOIN

Enunciado

Devolver un reporte completo de líneas con nombre del producto.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_join_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_nombre_producto TEXT;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;

    c_lineas CURSOR FOR
        SELECT lp.linea_id, lp.pedido_id, lp.producto_id,
               p.nombre, lp.cantidad, lp.precio_unit
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT ROW(
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit
        );
    END LOOP;

    CLOSE c_lineas;
END;
```

```
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_lineas_join_setof_record() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_nombre_producto TEXT,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

5. SETOF RECORD + cursores múltiples sincronizados

Enunciado

Recorrer pedidos y sus líneas con dos cursores y devolver un registro compuesto dinámico.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_lineas_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    c_lin CURSOR (p_id INTEGER) FOR
        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_id;

    v_pedido_id INTEGER;
    v_linea_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        OPEN c_lin(v_pedido_id);

        LOOP
            FETCH c_lin INTO v_linea_id, v_producto_id, v_cantidad;
            EXIT WHEN NOT FOUND;
```

```
            RETURN NEXT ROW(
                v_pedido_id,
                v_linea_id,
                v_producto_id,
                v_cantidad
            );
        END LOOP;

        CLOSE c_lin;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_pedidos_lineas_setof_record() AS t(
    v_pedido_id INTEGER,
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER
);
```

6. SETOF RECORD + cursores + JSONB

Enunciado

Devolver un JSON por cada pedido usando cursores y `SETOF RECORD`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_json_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    v_pedido_id INTEGER;
    v_json JSONB;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        v_json := (
            SELECT jsonb_agg(
                jsonb_build_object(
                    'linea_id', linea_id,
```

```
                    'producto_id', producto_id,
                    'cantidad', cantidad,
                    'precio_unit', precio_unit
                )
            )
            FROM tienda.lineas_pedido
            WHERE pedido_id = v_pedido_id
        );

        RETURN NEXT ROW(v_pedido_id, v_json);
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_pedidos_json_setof_record() AS t(
    v_pedido_id INTEGER,
    v_json JSONB
);
```

¿Qué módulo quieres ahora?

Puedo preparar:

- **Cursores ETL y batch jobs**
- **Cursores con SAVEPOINT avanzado**
- **Cursores con paralelismo simulado**
- **Cursores para auditoría y replicación**

Dime cuál quieres y te lo dejo perfecto con tu formato de variables.

el poner v_cursor es opcional ? en :RETURN NEXT v_cursor;
No, Jose — **no es opcional**. Y además, **RETURN NEXT v_cursor; es incorrecto en PostgreSQL**, porque un cursor **no es una fila**, sino un *puntero* a un conjunto de resultados.

Vamos a dejarlo **cerrado al 100%**, con precisión técnica y ejemplos.

Respuesta directa

No puedes hacer:

plpgsql

```
RETURN NEXT v_cursor;
```

Y tampoco es opcional:

plpgsql

```
RETURN NEXT;
```

No funciona con cursores.

Lo correcto es:

plpgsql

```
RETURN NEXT v_rec;
```

o

plpgsql

```
RETURN NEXT ROW(...);
```

Porque **RETURN NEXT siempre devuelve una fila**, no un cursor.

¿Por qué no se puede devolver un cursor con RETURN NEXT?

Porque:

- Un **cursor** es un *refcursor*, un identificador textual (ej. `'cur_lineas'`)
- No contiene datos, solo apunta a ellos
- `RETURN NEXT` **solo acepta filas**, no punteros

Si quieres devolver cursores, debes usar:

plpgsql

```
RETURNS SETOF refcursor
```

Y entonces sí:

plpgsql

```
RETURN NEXT v_cursor;
```

Pero eso es **otro caso**, donde la función devuelve **cursores**, no filas.

1. Caso válido: devolver cursores (SETOF refcursor)

Aquí **sí** se usa `RETURN NEXT v_cursor`.

plpgsql

```
CREATE OR REPLACE FUNCTION tienda.fn_varios_cursores()
RETURNS SETOF refcursor
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor;
BEGIN
    v_cursor := 'cur_clientes';
    OPEN v_cursor FOR SELECT * FROM tienda.clientes;
    RETURN NEXT v_cursor;

    v_cursor := 'cur_productos';
    OPEN v_cursor FOR SELECT * FROM tienda.productos;
    RETURN NEXT v_cursor;

    RETURN;
END;
$$;
```

Aquí `RETURN NEXT v_cursor` es correcto Porque el tipo de retorno es `SETOF refcursor`

2. Caso inválido: devolver filas con cursores

Si tu función es:

plpgsql

```
RETURNS SETOF RECORD
```

o

plpgsql

```
RETURNS TABLE(...)
```

Entonces **NO** puedes hacer:

plpgsql

```
RETURN NEXT v_cursor;
```

Porque un cursor **no es una fila**.

3. Caso correcto: devolver filas usando cursores

plpgsql

```
FETCH v_cursor INTO v_rec;
```

```
RETURN NEXT v_rec;
```

Ejemplo:

plpgsql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_rec RECORD;
    v_cursor refcursor := 'cur_lineas';
BEGIN
    OPEN v_cursor FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso:

sql

```
SELECT *
FROM tienda.fn_lineas_setof_record() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

4. Resumen final

¿Qué devuelvo?	¿Qué uso?	¿RETURN NEXT v_cursor?
Filas	`RETURNS SETOF RECORD` / `RETURNS TABLE`	No
Cursores	`RETURNS SETOF refcursor`	Sí
Una fila	`RETURN NEXT v_rec`	Sí
Muchas filas	`RETURN NEXT` dentro de un LOOP	Sí

Conclusión

Para devolver filas: `RETURN NEXT v_rec`

Si quieres, puedo prepararte:

- **Cómo devolver cursores y filas en la misma función**
- **Cursores avanzados con estructuras complejas**
- **Cursores + SETOF RECORD + SQL dinámico**

Dime cuál módulo quieres y te lo dejo perfecto con tu formato de variables.

v_rec es obligatorio en return next v_rec ?
Jose, vamos a dejar esto **cerrado de forma definitiva**, porque es una duda clave cuando trabajas con cursores y funciones `SETOF RECORD` en PostgreSQL:

Respuesta directa

Sí, es obligatorio poner una fila (un RECORD o un ROW) en `RETURN NEXT`**.** Pero **NO es obligatorio que se llame** `v_rec`. Lo obligatorio es **devolver una FILA**, no el nombre de la variable.

En otras palabras:

Obligatorio: devolver una fila

No obligatorio: que la variable se llame `v_rec`

Puedes devolver:

- `RETURN NEXT v_rec;`
- `RETURN NEXT ROW(v_linea_id, v_cantidad);`
- `RETURN NEXT v_registro;`
- `RETURN NEXT ROW(...);`

Pero **NO puedes** hacer:

- `RETURN NEXT;`
- `RETURN NEXT v_cursor;` (salvo `SETOF refcursor`)
- `RETURN NEXT NULL;` (no es una fila válida)

¿Por qué es obligatorio devolver una fila?

Porque:

`RETURN NEXT` siempre devuelve una fila

No puede devolver:

- cursores
- arrays
- JSON
- nada vacío
- un puntero
- un tipo no compuesto

Solo **una fila**.

1. Caso correcto: devolver un RECORD

plpgsql

```
DECLARE
    v_rec RECORD;
BEGIN
    FETCH v_cursor INTO v_rec;
    RETURN NEXT v_rec;
END;
```

Aquí `v_rec` es una fila → válido.

2. Caso correcto: devolver un ROW sin variable

plpgsql

```
RETURN NEXT ROW(v_linea_id, v_cantidad, v_precio_unit);
```

No usas `v_rec` → también válido.

3. Caso correcto: devolver un %ROWTYPE

plpgsql

```
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
BEGIN
```

```
    FETCH c_lineas INTO v_registro;
    RETURN NEXT v_registro;
END;
```

`v_registro` es una fila → válido.

4. Caso correcto: devolver un tipo compuesto

plpgsql

```
DECLARE
    v_reg tienda.tipo_linea;
BEGIN
    FETCH c_lineas INTO v_reg.v_linea_id, v_reg.v_cantidad;
    RETURN NEXT v_reg;
END;
```

`v_reg` es una fila → válido.

5. Caso correcto: devolver un cursor (solo si RETURNS SETOF refcursor)

plpgsql

```
RETURN NEXT v_cursor;
```

Solo válido si la función es:

plpgsql

```
RETURNS SETOF refcursor
```

6. Caso inválido: devolver nada

plpgsql

```
RETURN NEXT;
```

PostgreSQL no sabe qué fila devolver.

7. Caso inválido: devolver un cursor en funciones que devuelven filas

plpgsql

```
RETURN NEXT v_cursor;
```

Un cursor no es una fila.

Resumen final

¿Qué devuelve la función?	¿RETURN NEXT necesita fila?	¿v_rec obligatorio?
SETOF RECORD	Sí	No
RETURN TABLE	Sí	No
SETOF tipo compuesto	Sí	No
SETOF refcursor	No (devuelve cursores)	No

Obligatorio: devolver una fila

No obligatorio: que la variable se llame `v_rec`

Puedes usar cualquier nombre: `v_registro`, `v_row`, `v_linea`, `v_tmp`, etc.

Si quieres, puedo prepararte:

- **SETOF RECORD + SQL dinámico avanzado**
- **SETOF RECORD + cursores múltiples sincronizados**
- **SETOF RECORD + estructuras complejas (JSON, arrays, tipos)**

Dime cuál módulo quieres y te lo dejo perfecto con tu formato de variables.

entonces porque aqui si esta solamente RETURN NEXT; CREATE OR REPLACE FUNCTION tienda.fn_cursor_lineas_pedido(OUT v_linea_id INTEGER, OUT v_pedido_id INTEGER, OUT v_producto_id INTEGER, OUT v_cantidad INTEGER, OUT v_precio_unit NUMERIC) RETURNS SETOF RECORD LANGUAGE plpgsql AS $$ DECLARE c_lineas CURSOR FOR SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit FROM tienda.lineas_pedido ORDER BY linea_id; BEGIN OPEN c_lineas; LOOP FETCH c_lineas INTO v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit; EXIT WHEN NOT FOUND; RETURN NEXT; END LOOP; CLOSE c_lineas; END; $$;
Jose, **muy buena observación**, y aquí viene la explicación definitiva que aclara por qué en esa función aparece:

plpgsql

```
RETURN NEXT;
```

sin `v_rec`, sin `ROW(...)`, sin nada… y **funciona perfectamente**.

La clave es entender **cómo funcionan las funciones con OUT parameters**.

Respuesta directa

En funciones con parámetros OUT, RETURN NEXT; es totalmente válido.

No hace falta poner `RETURN NEXT v_rec;`

No hace falta poner `RETURN NEXT ROW(...);`

No hace falta poner ninguna variable

Porque los parámetros OUT ya son la fila que se devuelve.

1. ¿Por qué funciona RETURN NEXT solo?

Porque en funciones con OUT parameters, PostgreSQL crea **implícitamente un RECORD de salida** con esos OUT.

Ejemplo:

plpgsql

```
OUT v_linea_id INTEGER,
OUT v_pedido_id INTEGER,
OUT v_producto_id INTEGER,
OUT v_cantidad INTEGER,
OUT v_precio_unit NUMERIC
```

PostgreSQL internamente crea algo equivalente a:

plpgsql

```
DECLARE
    -- registro implícito
    _out_record RECORD;
```

Y cada OUT es un campo de ese registro:

Código

```
_out_record.v_linea_id
_out_record.v_pedido_id
_out_record.v_producto_id
_out_record.v_cantidad
_out_record.v_precio_unit
```

Cuando tú haces:

plpgsql

```
RETURN NEXT;
```

PostgreSQL interpreta:

> “Devuelve la fila formada por los OUT parameters actuales.”

2. Tu función funciona así:

plpgsql

```
FETCH c_lineas INTO
    v_linea_id,
    v_pedido_id,
    v_producto_id,
    v_cantidad,
    v_precio_unit;

RETURN NEXT;
```

Esto equivale a:

plpgsql

```
-- PostgreSQL internamente:
RETURN NEXT ROW(
    v_linea_id,
    v_pedido_id,
    v_producto_id,
    v_cantidad,
    v_precio_unit
);
```

Pero tú no lo escribes. PostgreSQL lo hace **automáticamente**.

3. Regla de oro

Si la función tiene OUT parameters → `RETURN NEXT;` es suficiente

Si la función NO tiene OUT parameters → `RETURN NEXT;` es inválido

Si la función devuelve SETOF RECORD sin OUT → debes usar `RETURN NEXT v_rec;` o `RETURN NEXT ROW(...)`

4. Comparación clara

Tipo de función	¿RETURN NEXT sin argumentos?	¿Por qué?
RETURNS SETOF RECORD con OUT parameters	Sí	OUT ya define la fila
RETURNS TABLE(...)	Sí	TABLE = OUT implícitos

Tipo de función	¿RETURN NEXT sin argumentos?	¿Por qué?
RETURNS SETOF RECORD sin OUT	No	No hay fila definida
RETURNS SETOF tipo_compuesto	No	Debes pasar la fila
RETURNS SETOF refcursor	Sí	Devuelves cursores, no filas

5. Ejemplo para verlo claro

Función con OUT → RETURN NEXT sin argumentos

plpgsql

```
CREATE OR REPLACE FUNCTION f1(
    OUT v_id INTEGER,
    OUT v_nombre TEXT
)
RETURNS SETOF RECORD
AS $$
BEGIN
    v_id := 1;
    v_nombre := 'Jose';
    RETURN NEXT;  -- válido
END;
$$ LANGUAGE plpgsql;
```

Función sin OUT → RETURN NEXT sin argumentos falla

plpgsql

```
CREATE OR REPLACE FUNCTION f2()
RETURNS SETOF RECORD
AS $$
BEGIN
    RETURN NEXT;  -- ERROR: no hay fila definida
END;
$$ LANGUAGE plpgsql;
```

6. Tu función es del primer tipo → por eso funciona

plpgsql

```
OUT v_linea_id INTEGER,
OUT v_pedido_id INTEGER,
OUT v_producto_id INTEGER,
OUT v_cantidad INTEGER,
OUT v_precio_unit NUMERIC
```

Estos OUT **son la fila**.

Por eso:

plpgsql

```
RETURN NEXT;
```

es correcto.

Resumen final

`RETURN NEXT;` es válido solo cuando la función tiene OUT parameters o RETURN TABLE.

En ese caso, los OUT parameters son la fila.

En funciones sin OUT, `RETURN NEXT;` es inválido.

No es obligatorio usar `v_rec`.

Lo obligatorio es devolver una fila, y con OUT ya la tienes.

Si quieres, puedo prepararte:

- **RETURNS TABLE vs OUT parameters**
- **SETOF RECORD + OUT + cursores avanzados**
- **Diseño profesional de funciones con OUT**

¿Quieres que prepare uno de esos módulos?

return table vs out postgresql
Aquí tienes, Jose — **la comparativa definitiva entre `RETURN TABLE` y parámetros `OUT` en PostgreSQL 18**, explicada con precisión técnica, ejemplos completos y tu formato de variables (`v_linea_id`, `v_pedido_id`, etc.). Incluyo Guided Links para profundizar:

- **Return Table**
- **OUT parameters**
- **SETOF RECORD**

Diferencia esencial (la frase que lo resume todo)

- `RETURN TABLE` define la estructura de salida **dentro de la firma de la función**. PostgreSQL ya sabe qué columnas devolver.
- `OUT parameters` también definen la estructura de salida, pero **cada OUT es una variable** que forma la fila de salida.

Ambos devuelven filas, pero **la forma de trabajar dentro de la función cambia**.

1. RETURN TABLE — estructura fija y clara

Características

- La estructura de salida se define en la firma.
- No necesitas declarar variables internas para esas columnas.
- Puedes usar `RETURN QUERY` o `RETURN NEXT`.
- Es más legible y moderno.
- Ideal para funciones tipo “vista”.

Ejemplo

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_return_table()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
    FROM tienda.lineas_pedido
    ORDER BY linea_id;
END;
$$;
```

Uso

sql

```
SELECT * FROM tienda.fn_lineas_return_table();
```

2. OUT parameters — estructura fija pero con variables internas

Características

- Cada OUT es una variable interna.
- La fila de salida se forma automáticamente con esos OUT.
- Puedes usar `RETURN NEXT;` sin argumentos.
- Muy útil con cursores explícitos.
- Ideal cuando necesitas manipular cada campo antes de devolverlo.

Ejemplo

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_out(
    OUT v_linea_id INTEGER,
    OUT v_pedido_id INTEGER,
    OUT v_producto_id INTEGER,
    OUT v_cantidad INTEGER,
    OUT v_precio_unit NUMERIC
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;  -- devuelve la fila formada por los OUT
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Uso

sql

```
SELECT * FROM tienda.fn_lineas_out();
```

3. ¿Por qué en OUT puedes usar `RETURN NEXT;` sin argumentos?

Porque los parámetros OUT **ya son la fila**.

PostgreSQL internamente construye un RECORD con los OUT:

Código

```
(v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit)
```

Por eso:

sql

```
RETURN NEXT;
```

equivale a:

sql

```
RETURN NEXT ROW(
    v_linea_id,
    v_pedido_id,
    v_producto_id,
    v_cantidad,
    v_precio_unit
);
```

4. Comparativa técnica

Característica	RETURN TABLE	OUT parameters
Define estructura	En la firma	En la firma
Variables internas	No necesarias	Sí, cada OUT es una variable
RETURN NEXT	Necesita fila explícita	Puede ser vacío (`RETURN NEXT;`)
RETURN QUERY	Muy cómodo	También funciona
Legibilidad		
Ideal para	Vistas dinámicas	Cursores, lógica fila a fila
Flexibilidad	Media	Alta
Performance	Igual	Igual

5. Ejemplo comparativo real

A) RETURN TABLE

sql

```
RETURN QUERY
SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
FROM tienda.lineas_pedido;
```

B) OUT parameters

sql

```
FETCH c_lineas INTO
    v_linea_id,
    v_pedido_id,
    v_producto_id,
    v_cantidad,
    v_precio_unit;

RETURN NEXT;
```

6. ¿Cuál usar en cada caso?

Usa RETURN TABLE cuando:

- La estructura es fija.
- Quieres claridad y menos código.
- Vas a devolver directamente un SELECT.
- No necesitas manipular cada campo.

Usa OUT parameters cuando:

- Necesitas cursores explícitos.
- Necesitas lógica fila a fila.
- Quieres modificar valores antes de devolverlos.
- Necesitas devolver filas parcialmente calculadas.

7. Ejemplo avanzado: misma función en ambos estilos

A) RETURN TABLE

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_return_table()
RETURNS TABLE (
    v_pedido_id INTEGER,
    v_total NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT pedido_id,
           (SELECT SUM(cantidad * precio_unit)
            FROM tienda.lineas_pedido lp
            WHERE lp.pedido_id = p.pedido_id)
    FROM tienda.pedidos p;
END;
$$;
```

B) OUT parameters

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_totales_out(
    OUT v_pedido_id INTEGER,
    OUT v_total NUMERIC
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
```

```
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        SELECT SUM(cantidad * precio_unit)
        INTO v_total
        FROM tienda.lineas_pedido
        WHERE pedido_id = v_pedido_id;

        RETURN NEXT;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

¿Quieres que prepare ahora…?

- **SETOF RECORD + cursores avanzados**
- **Cursores + SQL dinámico + OUT**
- **Diseño profesional de funciones para APIs SQL**

Dime cuál módulo quieres y te lo dejo perfecto con tu formato de variables.

cursores setof record postgresql
Perfecto, Jose. Vamos a construir **EL MÓDULO DEFINITIVO DE FUNCIONES** `SETOF RECORD` **con CURSORES en PostgreSQL 18**, con ejemplos avanzados, claros, y usando SIEMPRE tu formato de variables (`v_linea_id`, `v_pedido_id`, `v_producto_id`, etc.).
Este módulo es el más técnico de todos porque combina:

- Cursores explícitos
- Cursores dinámicos
- RECORD dinámico
- OUT parameters
- RETURN NEXT
- RETURN QUERY
- Estructuras complejas

Incluyo Guided Links para profundizar:

- **SETOF RECORD**
- **Cursores explícitos**
- **Cursores dinámicos**

- **Record dinámico**

1. Concepto clave: ¿Qué es `SETOF RECORD`?

`SETOF RECORD` permite devolver **filas con estructura dinámica**, es decir:

- La función NO define las columnas de salida.
- El usuario define la estructura al llamar la función.
- Es ideal para SQL dinámico, cursores y tablas variables.

Ejemplo de llamada:

sql

```
SELECT *
FROM tienda.fn_algo()
    AS t(v_col1 INTEGER, v_col2 TEXT, v_col3 NUMERIC);
```

2. Cursor explícito + SETOF RECORD (patrón base)

Enunciado

Recorrer líneas de pedido con un cursor explícito y devolverlas como `SETOF RECORD`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;

    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
```

```
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT ROW(
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit
        );
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_lineas_setof_record() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

3. SETOF RECORD + OUT parameters + cursores

Enunciado

Usar OUT parameters para que `RETURN NEXT;` funcione sin argumentos.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_out_setof_record(
    OUT v_linea_id INTEGER,
    OUT v_pedido_id INTEGER,
    OUT v_producto_id INTEGER,
    OUT v_cantidad INTEGER,
    OUT v_precio_unit NUMERIC
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RETURN NEXT;  -- OUT parameters ya forman la fila
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Uso

sql

```
SELECT * FROM tienda.fn_lineas_out_setof_record();
```

4. Cursor dinámico + SETOF RECORD (tabla variable)

Enunciado

Devolver cualquier tabla de la base de datos usando SQL dinámico y cursores.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_tabla_setof_record(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;
```

```
    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_tabla_setof_record('productos')
    AS t(
        v_producto_id INTEGER,
        v_nombre TEXT,
        v_precio NUMERIC,
        v_stock INTEGER,
        v_activo BOOLEAN
    );
```

5. SETOF RECORD + cursores múltiples sincronizados

Enunciado

Recorrer pedidos y sus líneas con dos cursores y devolver un registro compuesto dinámico.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_lineas_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    c_lin CURSOR (p_id INTEGER) FOR
        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_id;

    v_pedido_id INTEGER;
    v_linea_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c_ped;

    LOOP
```

```
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        OPEN c_lin(v_pedido_id);

        LOOP
            FETCH c_lin INTO v_linea_id, v_producto_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            RETURN NEXT ROW(
                v_pedido_id,
                v_linea_id,
                v_producto_id,
                v_cantidad
            );
        END LOOP;

        CLOSE c_lin;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_pedidos_lineas_setof_record() AS t(
    v_pedido_id INTEGER,
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER
);
```

6. SETOF RECORD + cursores + JSONB

Enunciado

Devolver un JSON por cada pedido usando cursores y `SETOF RECORD`.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_json_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    v_pedido_id INTEGER;
    v_json JSONB;
BEGIN
    OPEN c_ped;
```

```
    LOOP
        FETCH c_ped INTO v_pedido_id;
        EXIT WHEN NOT FOUND;

        v_json := (
            SELECT jsonb_agg(
                jsonb_build_object(
                    'linea_id', linea_id,
                    'producto_id', producto_id,
                    'cantidad', cantidad,
                    'precio_unit', precio_unit
                )
            )
            FROM tienda.lineas_pedido
            WHERE pedido_id = v_pedido_id
        );

        RETURN NEXT ROW(v_pedido_id, v_json);
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_pedidos_json_setof_record() AS t(
    v_pedido_id INTEGER,
    v_json JSONB
);
```

7. SETOF RECORD + cursores + lógica compleja (JOIN + cálculos)

Enunciado

Devolver un reporte completo de líneas con nombre del producto y total calculado.

Función

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_completo_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_nombre_producto TEXT;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;
```

```
    v_total_linea NUMERIC;

    c_lineas CURSOR FOR
        SELECT lp.linea_id, lp.pedido_id, lp.producto_id,
               p.nombre, lp.cantidad, lp.precio_unit
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        v_total_linea := v_cantidad * v_precio_unit;

        RETURN NEXT ROW(
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit,
            v_total_linea
        );
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_lineas_completo_setof_record() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_nombre_producto TEXT,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC
);
```

CURSORES IMPLICITOS

Los cursores implícitos en PostgreSQL son uno de los mecanismos más elegantes y potentes de PL/pgSQL porque te

permiten recorrer filas sin declarar cursores, sin `OPEN`, sin `FETCH`, sin `CLOSE`. PostgreSQL hace todo eso por ti automáticamente.

Aquí tienes el **módulo completo**, con ejemplos avanzados, claros y usando tu estilo de variables (`v_linea_id`, `v_pedido_id`, etc.). Incluyo Guided Links para profundizar:

- **Cursores implícitos**
- **Cursores explícitos**
- **SETOF RECORD**

Idea clave

Un cursor implícito aparece **siempre que usas un** `FOR r IN SELECT ... LOOP`.

PostgreSQL crea un cursor interno, lo abre, hace `FETCH` fila a fila y lo cierra al final. Tú solo escribes el bucle.

1. Cursor implícito básico

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_cursor_implicito()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        RAISE NOTICE 'Línea %, Pedido %, Producto %, Cantidad %, Precio %',
            v_registro.linea_id,
            v_registro.pedido_id,
            v_registro.producto_id,
            v_registro.cantidad,
            v_registro.precio_unit;
    END LOOP;
END;
$$;
```

¿Qué hace PostgreSQL internamente?

- Declara un cursor
- Lo abre
- Hace `FETCH` fila a fila

- Detecta `NOT FOUND`
- Lo cierra

2. Cursor implícito con variables individuales

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_cursor_implicito_vars()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;
BEGIN
    FOR v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        RAISE NOTICE 'Línea %, Cantidad %', v_linea_id, v_cantidad;
    END LOOP;
END;
$$;
```

3. Cursor implícito + RETURN TABLE

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_return_table_implicito()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Aquí `RETURN NEXT;` funciona porque `RETURN TABLE` crea OUT implícitos.

4. Cursor implícito + SETOF RECORD

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_setof_record_implicito()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        RETURN NEXT v_registro;
    END LOOP;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_lineas_setof_record_implicito() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
);
```

5. Cursor implícito + JOIN + cálculos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_join_implicito()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_nombre_producto TEXT,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id,
        v_pedido_id,
        v_producto_id,
```

```
        v_nombre_producto,
        v_cantidad,
        v_precio_unit,
        v_total_linea
    IN
        SELECT
            lp.linea_id,
            lp.pedido_id,
            lp.producto_id,
            p.nombre,
            lp.cantidad,
            lp.precio_unit,
            lp.cantidad * lp.precio_unit AS total_linea
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

6. Cursor implícito + SQL dinámico

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_implicito_sql_dinamico(
    p_stock_min INTEGER
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
    v_sql TEXT;
BEGIN
    v_sql := format(
        'SELECT producto_id, nombre, precio, stock
         FROM tienda.productos
         WHERE stock >= %s
         ORDER BY producto_id',
        p_stock_min
    );

    FOR v_registro IN EXECUTE v_sql
    LOOP
        RETURN NEXT v_registro;
    END LOOP;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_implicito_sql_dinamico(20)
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER);
```

7. Cursor implícito + lógica compleja por fila

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_implicito_logica()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id
    LOOP
        IF v_registro.cantidad > 10 THEN
            RAISE NOTICE 'Línea % tiene cantidad alta: %',
                v_registro.linea_id, v_registro.cantidad;
        END IF;
    END LOOP;
END;
$$;
```

8. Cursor implícito + manejo de errores

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_implicito_errores()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id
    LOOP
        BEGIN
            IF v_registro.cantidad < 0 THEN
                RAISE EXCEPTION 'Cantidad negativa en línea %',
v_registro.linea_id;
            END IF;

            RAISE NOTICE 'OK línea %', v_registro.linea_id;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error en línea %: %', v_registro.linea_id,
SQLERRM;
        END;
    END LOOP;
END;
$$;
```

CURSORES EXPLICITOS

Idea clave

Un **cursor explícito** es un cursor que tú controlas manualmente:

- Lo declaras
- Lo abres
- Haces `FETCH`
- Haces `MOVE`
- Lo cierras

Es ideal para:

- Procesamiento fila a fila
- ETL
- SQL dinámico
- Lógica compleja
- SCROLL (ir adelante/atrás)
- Devolver cursores a clientes externos

1. Cursor explícito básico

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_explicito()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;

    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Pedido %, Producto %, Cantidad %, Precio %',
            v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit;
    END LOOP;
```

```
    CLOSE c_lineas;
END;
$$;
```

2. Cursor explícito con parámetros

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_param(
    p_activo BOOLEAN
)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c_prod CURSOR (estado BOOLEAN) FOR
        SELECT producto_id, nombre, precio, stock
        FROM tienda.productos
        WHERE activo = estado;

    v_producto_id INTEGER;
    v_nombre TEXT;
    v_precio NUMERIC;
    v_stock INTEGER;
BEGIN
    OPEN c_prod(p_activo);

    LOOP
        FETCH c_prod INTO v_producto_id, v_nombre, v_precio, v_stock;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Producto %, Nombre %, Precio %, Stock %',
            v_producto_id, v_nombre, v_precio, v_stock;
    END LOOP;

    CLOSE c_prod;
END;
$$;
```

3. Cursor SCROLL (avanzado)

Permite moverte hacia adelante y atrás.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_scroll()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c_ped SCROLL CURSOR FOR
        SELECT pedido_id, cliente_id
        FROM tienda.pedidos
        ORDER BY pedido_id;

    v_pedido_id INTEGER;
```

```
    v_cliente_id INTEGER;
BEGIN
    OPEN c_ped;

    FETCH NEXT FROM c_ped INTO v_pedido_id, v_cliente_id;
    RAISE NOTICE 'Primero: Pedido %, Cliente %', v_pedido_id, v_cliente_id;

    FETCH PRIOR FROM c_ped INTO v_pedido_id, v_cliente_id;
    RAISE NOTICE 'Volver atrás: Pedido %, Cliente %', v_pedido_id, v_cliente_id;

    CLOSE c_ped;
END;
$$;
```

4. Cursor explícito + SQL dinámico

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_sql_dinamico(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_rec RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_rec;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_rec;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_sql_dinamico('productos')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

5. Cursor explícito + JOIN + cálculos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_join()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT lp.linea_id, lp.pedido_id, lp.producto_id,
               p.nombre, lp.cantidad, lp.precio_unit
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id;

    v_linea_id INTEGER;
    v_pedido_id INTEGER;
    v_producto_id INTEGER;
    v_nombre_producto TEXT;
    v_cantidad INTEGER;
    v_precio_unit NUMERIC;
    v_total_linea NUMERIC;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit;

        EXIT WHEN NOT FOUND;

        v_total_linea := v_cantidad * v_precio_unit;

        RETURN NEXT ROW(
            v_linea_id,
            v_pedido_id,
            v_producto_id,
            v_nombre_producto,
            v_cantidad,
            v_precio_unit,
            v_total_linea
        );
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

6. Cursor explícito + manejo de errores

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_errores()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
```

```
    c_lineas CURSOR FOR
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c_lineas;

    LOOP
        BEGIN
            FETCH c_lineas INTO v_linea_id, v_cantidad;
            EXIT WHEN NOT FOUND;

            IF v_cantidad < 0 THEN
                RAISE EXCEPTION 'Cantidad negativa en línea %', v_linea_id;
            END IF;

            RAISE NOTICE 'OK línea %', v_linea_id;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error en línea %: %', v_linea_id, SQLERRM;
        END;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

7. Cursor explícito + SAVEPOINT (transacciones largas)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_savepoint()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, cantidad FROM tienda.lineas_pedido ORDER BY linea_id;

    v_linea_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_linea_id, v_cantidad;
        EXIT WHEN NOT FOUND;

        SAVEPOINT sp_linea;

        BEGIN
            IF v_cantidad = 0 THEN
                RAISE EXCEPTION 'Cantidad cero en línea %', v_linea_id;
            END IF;
```

```
            RAISE NOTICE 'Procesada línea %', v_linea_id;

        EXCEPTION
            WHEN others THEN
                RAISE NOTICE 'Error en línea %, rollback parcial: %',
                    v_linea_id, SQLERRM;
                ROLLBACK TO SAVEPOINT sp_linea;
        END;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

8. Cursor explícito + devolver cursores (SETOF refcursor)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_varios_cursores()
RETURNS SETOF refcursor
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor;
BEGIN
    v_cursor := 'cur_clientes';
    OPEN v_cursor FOR SELECT * FROM tienda.clientes;
    RETURN NEXT v_cursor;

    v_cursor := 'cur_productos';
    OPEN v_cursor FOR SELECT * FROM tienda.productos;
    RETURN NEXT v_cursor;

    RETURN;
END;
$$;
```

esto v_registro tienda.lineas_pedido%ROWTYPE; equivale a esto v_registro record. ?

Jose, vamos a dejar esto **cerrado al 100%**, porque esta duda es clave cuando trabajas con cursores, `%ROWTYPE` y `RECORD` en PostgreSQL:

Respuesta directa

Sí y no. `v_registro tienda.lineas_pedido%ROWTYPE` **y** `v_registro RECORD` **no son equivalentes**, pero **pueden comportarse igual** en muchos casos.

La diferencia es **muy importante**:

1. `%ROWTYPE` → estructura FIJA

Cuando declaras:

plpgsql

```
v_registro tienda.lineas_pedido%ROWTYPE;
```

Estás diciendo:

> "v_registro tiene **exactamente** los campos de la tabla `tienda.lineas_pedido`."

Es un tipo **fijo**, con columnas conocidas:

- `v_registro.linea_id`
- `v_registro.pedido_id`
- `v_registro.producto_id`
- `v_registro.cantidad`
- `v_registro.precio_unit`

Ventajas

- Tipado fuerte
- Autocompletado en IDE
- Seguridad: no puedes meter columnas que no existen
- Más rápido para el compilador

Limitación

- Solo sirve para esa tabla
- No puedes cambiar la estructura dinámicamente

2. `RECORD` → estructura DINÁMICA

Cuando declaras:

plpgsql

```
v_registro RECORD;
```

Estás diciendo:

> "v_registro tendrá la estructura del SELECT que le metas."

Es **dinámico**: La estructura depende del `SELECT`, del cursor, o del `EXECUTE`.

Ventajas

- Flexible
- Perfecto para SQL dinámico
- Perfecto para cursores dinámicos
- Perfecto para funciones SETOF RECORD

Limitación

- No tiene autocompletado
- Si cambias el SELECT, cambian los campos
- Si haces un SELECT con columnas distintas, cambia la estructura

3. Comparación clara

Característica	`%ROWTYPE`	`RECORD`
Estructura	Fija	Dinámica
Basado en	Una tabla o tipo	El SELECT que recibe
Autocompletado	Sí	No
Seguridad de tipos	Alta	Baja
Ideal para	Tablas conocidas	SQL dinámico, cursores
Cambia según SELECT	No	Sí
Uso con cursores	Excelente	Excelente

4. Ejemplo donde NO son equivalentes

`%ROWTYPE` falla:

plpgsql

```
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
BEGIN
    SELECT linea_id, cantidad
    INTO v_registro
    FROM tienda.lineas_pedido;
END;
```

ERROR: faltan columnas Porque `%ROWTYPE` exige **todas** las columnas.

`RECORD` funciona:

plpgsql

```
DECLARE
    v_registro RECORD;
BEGIN
    SELECT linea_id, cantidad
    INTO v_registro
```

```
    FROM tienda.lineas_pedido;
END;
```

Perfecto Porque `RECORD` se adapta al SELECT.

5. Ejemplo donde sí se comportan igual

plpgsql

```
DECLARE
    v_registro RECORD;
BEGIN
    SELECT * INTO v_registro
    FROM tienda.lineas_pedido
    WHERE linea_id = 1;
END;
```

Aquí `RECORD` tendrá **exactamente** los mismos campos que `%ROWTYPE`.

Resumen definitivo

No equivalen técnicamente

`%ROWTYPE` = estructura fija `RECORD` = estructura dinámica

Pero pueden comportarse igual

Si haces `SELECT *` sobre la misma tabla.

Regla de oro

- Si la estructura es **siempre la misma** → usa **%ROWTYPE**
- Si la estructura **puede cambiar** → usa **RECORD**
- Si usas **SQL dinámico** → usa **RECORD**
- Si usas **cursores dinámicos** → usa **RECORD**
- Si usas **cursores fijos** → cualquiera sirve

¿Quieres que te prepare un módulo sobre cuándo usar %ROWTYPE vs RECORD en cursores?

Puedo hacer:

- **Cursores + %ROWTYPE**
- **Cursores + RECORD dinámico**
- **Cursores + SQL dinámico + RECORD**

Dime cuál quieres y te lo dejo perfecto con tu estilo.

Hoy
cursores rowtype postgresql
Perfecto, Jose. Aquí tienes **el módulo definitivo de CURSORES EXPLÍCITOS con %ROWTYPE en PostgreSQL**, explicado con precisión técnica, ejemplos avanzados y tu estilo de variables. Este es el patrón profesional para recorrer tablas con cursores y aprovechar %ROWTYPE para tener **tipado fuerte**, autocompletado y seguridad.

Incluyo Guided Links para profundizar:

- **Cursores explícitos**
- **Cursores implícitos**
- **Rowtype**
- **Record dinámico**

Qué es %ROWTYPE en cursores explícitos

%ROWTYPE crea una variable con **la estructura exacta de la tabla**. Esto significa:

- Campos fijos
- Tipado fuerte
- Seguridad total
- Autocompletado en IDE
- No depende del SELECT (si cambias columnas, falla)

Es perfecto para cursores explícitos cuando **siempre** vas a recorrer la misma tabla.

1. Cursor explícito + %ROWTYPE (patrón base)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;

    c_lineas CURSOR FOR
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Pedido %, Producto %, Cantidad %, Precio %',
            v_registro.linea_id,
            v_registro.pedido_id,
```

```
            v_registro.producto_id,
            v_registro.cantidad,
            v_registro.precio_unit;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

2. Cursor explícito + %ROWTYPE + lógica por fila

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype_logica()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;

    c_lineas CURSOR FOR
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        IF v_registro.cantidad > 10 THEN
            RAISE NOTICE 'Línea % tiene cantidad alta: %',
                v_registro.linea_id, v_registro.cantidad;
        END IF;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

3. Cursor explícito + %ROWTYPE + cálculos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype_totales()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    v_total NUMERIC;
BEGIN
    FOR v_registro IN
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id
    LOOP
        v_total := v_registro.cantidad * v_registro.precio_unit;

        RETURN NEXT ROW(
```

```
            v_registro.linea_id,
            v_registro.pedido_id,
            v_registro.producto_id,
            v_registro.cantidad,
            v_registro.precio_unit,
            v_total
        );
    END LOOP;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_rowtype_totales() AS t(
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total NUMERIC
);
```

4. Cursor explícito + %ROWTYPE + JOIN

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype_join()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    v_nombre_producto TEXT;

    c_lineas CURSOR FOR
        SELECT lp.*, p.nombre
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro, v_nombre_producto;
        EXIT WHEN NOT FOUND;

        RETURN NEXT ROW(
            v_registro.linea_id,
            v_registro.pedido_id,
            v_registro.producto_id,
            v_nombre_producto,
            v_registro.cantidad,
            v_registro.precio_unit
        );
```

```
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

5. Cursor explícito + %ROWTYPE vs RECORD (comparativa real)

Caso	%ROWTYPE	RECORD
SELECT * de la misma tabla	Perfecto	Perfecto
SELECT con columnas distintas	Falla	Funciona
SQL dinámico	No sirve	Ideal
JOIN con columnas adicionales	Falla	Funciona
Autocompletado	Sí	No
Seguridad de tipos	Alta	Baja

6. Cursor explícito + %ROWTYPE + SCROLL

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_rowtype_scroll()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.lineas_pedido%ROWTYPE;
    c_lineas SCROLL CURSOR FOR
        SELECT * FROM tienda.lineas_pedido ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    FETCH NEXT FROM c_lineas INTO v_registro;
    RAISE NOTICE 'Primera línea: %', v_registro.linea_id;

    FETCH PRIOR FROM c_lineas INTO v_registro;
    RAISE NOTICE 'Volver atrás: %', v_registro.linea_id;

    CLOSE c_lineas;
END;
$$;
```

Resumen definitivo

%ROWTYPE es perfecto para cursores explícitos cuando:

- Siempre recorres la misma tabla
- Quieres seguridad de tipos
- Quieres autocompletado
- Quieres evitar RECORD dinámico

`%ROWTYPE` NO sirve cuando:

- El SELECT cambia columnas
- Usas SQL dinámico
- Haces JOIN con columnas adicionales
- La estructura es variable

En esos casos debes usar **RECORD**.

¿Quieres ahora el módulo de cursores + RECORD dinámico?

Es el complemento perfecto de este.

si

Perfecto, Jose. Aquí tienes **el módulo completo de CURSORES + `RECORD` dinámico en PostgreSQL**, explicado con precisión técnica, ejemplos avanzados y tu estilo de variables. Este módulo es el complemento natural del de `%ROWTYPE`: ahora verás **cuándo usar** `RECORD`, cómo aprovechar su flexibilidad y cómo combinarlo con cursores explícitos, SQL dinámico y estructuras variables.

Incluyo Guided Links para profundizar:

- **Cursores explícitos**
- **Record dinámico**
- **SQL dinámico**
- **SETOF RECORD**

Qué es `RECORD` en cursores

`RECORD` es un tipo **dinámico**: su estructura depende del `SELECT`, del cursor o del `EXECUTE`.

Esto lo hace perfecto para:

- SELECT con columnas variables
- SQL dinámico
- JOIN con columnas adicionales
- cursores que cambian según parámetros
- funciones SETOF RECORD
- recorrer tablas distintas con la misma función

1. Cursor explícito + RECORD (patrón base)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;

    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, cantidad
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Pedido %, Cantidad %',
            v_registro.linea_id,
            v_registro.pedido_id,
            v_registro.cantidad;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

2. Cursor explícito + RECORD + JOIN (estructura variable)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_join()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;

    c_lineas CURSOR FOR
        SELECT lp.linea_id, lp.pedido_id, lp.producto_id,
               p.nombre AS nombre_producto,
               lp.cantidad, lp.precio_unit
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
        ORDER BY lp.linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Línea %, Producto %, Cantidad %',
```

```
            v_registro.linea_id,
            v_registro.nombre_producto,
            v_registro.cantidad;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

3. Cursor explícito + RECORD + SQL dinámico (caso profesional)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_sql_dinamico(
    p_tabla TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_cursor refcursor := 'cur_dyn';
    v_registro RECORD;
BEGIN
    v_sql := format('SELECT * FROM tienda.%I ORDER BY 1', p_tabla);

    OPEN v_cursor FOR EXECUTE v_sql;

    LOOP
        FETCH v_cursor INTO v_registro;
        EXIT WHEN NOT FOUND;

        RETURN NEXT v_registro;
    END LOOP;

    CLOSE v_cursor;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_record_sql_dinamico('productos')
    AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER, v_activo BOOLEAN);
```

4. Cursor explícito + RECORD + lógica compleja por fila

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_logica()
```

```
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;

    c_lineas CURSOR FOR
        SELECT linea_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_registro;
        EXIT WHEN NOT FOUND;

        IF v_registro.cantidad * v_registro.precio_unit > 100 THEN
            RAISE NOTICE 'Línea % supera 100€', v_registro.linea_id;
        END IF;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

5. Cursor explícito + RECORD + múltiples cursores sincronizados

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_cursor_record_multiples()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_pedido RECORD;
    v_linea RECORD;

    c_ped CURSOR FOR SELECT pedido_id FROM tienda.pedidos ORDER BY pedido_id;
    c_lin CURSOR (p_id INTEGER) FOR
        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_id;
BEGIN
    OPEN c_ped;

    LOOP
        FETCH c_ped INTO v_pedido;
        EXIT WHEN NOT FOUND;

        OPEN c_lin(v_pedido.pedido_id);

        LOOP
            FETCH c_lin INTO v_linea;
            EXIT WHEN NOT FOUND;

            RETURN NEXT ROW(
                v_pedido.pedido_id,
```

```
                v_linea.linea_id,
                v_linea.producto_id,
                v_linea.cantidad
            );
        END LOOP;

        CLOSE c_lin;
    END LOOP;

    CLOSE c_ped;
END;
$$;
```

Uso

sql

```
SELECT *
FROM tienda.fn_cursor_record_multiples() AS t(
    v_pedido_id INTEGER,
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER
);
```

6. Comparativa %ROWTYPE vs RECORD en cursores

Caso	%ROWTYPE	RECORD
SELECT * de la misma tabla	Perfecto	Perfecto
SELECT con columnas distintas	Falla	Funciona
JOIN con columnas adicionales	Falla	Funciona
SQL dinámico	No sirve	Ideal
Autocompletado	Sí	No
Seguridad de tipos	Alta	Baja
Flexibilidad	Baja	Alta

Resumen definitivo

Usa %ROWTYPE cuando:

- Siempre recorres la misma tabla
- Quieres seguridad de tipos
- Quieres autocompletado
- No usas SQL dinámico

Usa RECORD cuando:

- La estructura del SELECT cambia
- Usas SQL dinámico

- Haces JOIN con columnas adicionales
- Necesitas flexibilidad total
- La función devuelve SETOF RECORD

Todas las variantes de return table con variables transformadas, cursores implicitos, explicitos

Aquí tienes, Jose — **el catálogo definitivo de TODAS las variantes posibles de** `RETURN TABLE` **combinadas con:**

- Cursores implícitos
- Cursores explícitos
- Variables transformadas
- `%ROWTYPE`
- `RECORD`
- Sin `RECORD`
- SQL dinámico
- JOIN + cálculos

Todo en un solo módulo, limpio, ordenado y profesional, con tu estilo de variables (`v_linea_id`, `v_pedido_id`, etc.). Incluyo Guided Links para profundizar:

- **Return Table**
- **Cursores implícitos**
- **Cursores explícitos**
- **Record dinámico**
- **Rowtype**

1. RETURN TABLE + SELECT directo (sin cursores)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_directo()
RETURNS TABLE (
```

```
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
    FROM tienda.lineas_pedido;
END;
$$;
```

2. RETURN TABLE + cursores implícitos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_implicito()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

3. RETURN TABLE + cursores implícitos + variables transformadas

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_implicito_transformado()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id, v_total_linea IN
        SELECT linea_id, cantidad * precio_unit
        FROM tienda.lineas_pedido
```

```
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

4. RETURN TABLE + cursores explícitos

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_explicito()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido;

BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO
            v_linea_id, v_pedido_id, v_producto_id, v_cantidad, v_precio_unit;
        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

5. RETURN TABLE + cursores explícitos + variables transformadas

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_explicito_transformado()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_cantidad numeric,
    v_precio_unit numeric,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
```

```
    c_lineas CURSOR FOR
        SELECT linea_id, cantidad, precio_unit
        FROM tienda.lineas_pedido;

BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_linea_id, v_cantidad, v_precio_unit;
        EXIT WHEN NOT FOUND;

        v_total_linea := v_cantidad * v_precio_unit;

        RETURN NEXT;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

6. RETURN TABLE + %ROWTYPE (cursor explícito)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_rowtype()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg tienda.lineas_pedido%ROWTYPE;
    c_lineas CURSOR FOR SELECT * FROM tienda.lineas_pedido;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_reg;
        EXIT WHEN NOT FOUND;

        v_linea_id := v_reg.linea_id;
        v_pedido_id := v_reg.pedido_id;
        v_producto_id := v_reg.producto_id;
        v_cantidad := v_reg.cantidad;
        v_precio_unit := v_reg.precio_unit;

        RETURN NEXT;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

7. RETURN TABLE + RECORD (cursor explícito)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_record()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg RECORD;
    c_lineas CURSOR FOR
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido;
BEGIN
    OPEN c_lineas;

    LOOP
        FETCH c_lineas INTO v_reg;
        EXIT WHEN NOT FOUND;

        v_linea_id := v_reg.linea_id;
        v_pedido_id := v_reg.pedido_id;
        v_producto_id := v_reg.producto_id;
        v_cantidad := v_reg.cantidad;
        v_precio_unit := v_reg.precio_unit;

        RETURN NEXT;
    END LOOP;

    CLOSE c_lineas;
END;
$$;
```

8. RETURN TABLE + RECORD + SQL dinámico

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_record_sql(p_tabla TEXT)
RETURNS TABLE (
    v_id INTEGER,
    v_nombre TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg RECORD;
    v_sql TEXT;
```

```
BEGIN
    v_sql := format('SELECT id, nombre FROM tienda.%I ORDER BY id', p_tabla);

    FOR v_reg IN EXECUTE v_sql
    LOOP
        v_id := v_reg.id;
        v_nombre := v_reg.nombre;

        RETURN NEXT;
    END LOOP;
END;
$$;
```

9. RETURN TABLE + JOIN + transformaciones

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_join_transformado()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_nombre_producto TEXT,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR v_linea_id, v_nombre_producto, v_total_linea IN
        SELECT lp.linea_id,
               p.nombre,
               lp.cantidad * lp.precio_unit
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Resumen final (catálogo completo)

Variante	Implícito	Explícito	RECORD	%ROWTYPE	Transformaciones	SQL dinámico
RETURN TABLE directo	—	—	—	—	—	—
RETURN TABLE implícito		—	—	—		—
RETURN TABLE explícito	—		—	—		—
RETURN	—			—		

Variante	Implícito	Explícito	RECORD	%ROWTYPE	Transformaciones	SQL dinámico
TABLE + RECORD RETURN						
TABLE + ROWTYPE	—	—				—
RETURN TABLE + JOIN			—			—

Si quieres, puedo prepararte ahora:

- **TODAS las variantes de SETOF RECORD**
- **TODAS las variantes de OUT parameters**
- **TODAS las variantes de cursores SCROLL, MOVE, FETCH ABSOLUTE**

Solo dime cuál módulo quieres.

ahora return table + record pero con cursor implicito ,
Aquí lo tienes, Jose — **la variante exacta que pediste:** `RETURN TABLE` + `RECORD` + ***cursor implícito***. Esta es una combinación muy específica y elegante: usar un cursor implícito (`FOR v_reg IN SELECT ... LOOP`) pero devolver filas mediante **RETURN TABLE**, asignando los valores desde un **RECORD dinámico**.

Incluyo Guided Links para profundizar:

- **Return Table**
- **Record dinámico**
- **Cursores implícitos**

RETURN TABLE + RECORD + cursor implícito

Esta variante usa:

- `RETURNS TABLE(...)` → estructura fija
- `v_reg RECORD` → estructura dinámica del SELECT
- Cursor implícito → `FOR v_reg IN SELECT ... LOOP`
- `RETURN NEXT` → devuelve cada fila usando las columnas del TABLE

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_record_implicito()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_pedido_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC
)
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg RECORD;
BEGIN
    FOR v_reg IN
        SELECT linea_id, pedido_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        ORDER BY linea_id
    LOOP
        v_linea_id   := v_reg.linea_id;
        v_pedido_id  := v_reg.pedido_id;
        v_producto_id:= v_reg.producto_id;
        v_cantidad   := v_reg.cantidad;
        v_precio_unit:= v_reg.precio_unit;

        RETURN NEXT;
    END LOOP;
END;
$$;
```

Uso

sql

```
SELECT * FROM tienda.fn_rt_record_implicito();
```

¿Por qué esta variante es especial?

Porque combina:

- **RETURN TABLE** → estructura fija y clara
- **RECORD** → SELECT flexible, dinámico
- **Cursor implícito** → sin OPEN/FETCH/CLOSE
- **RETURN NEXT** → devuelve cada fila del TABLE

Es perfecta cuando:

- Quieres devolver una tabla fija
- Pero el SELECT puede cambiar columnas, filtros, joins
- Y no quieres usar cursores explícitos

Versión con variables transformadas (misma variante, más avanzada)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_record_implicito_transformado()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg RECORD;
BEGIN
    FOR v_reg IN
        SELECT linea_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
    LOOP
        v_linea_id   := v_reg.linea_id;
        v_total_linea:= v_reg.cantidad * v_reg.precio_unit;

        RETURN NEXT;
    END LOOP;
END;
$$;
```

Versión con JOIN (cursor implícito + RECORD + transformaciones)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_rt_record_implicito_join()
RETURNS TABLE (
    v_linea_id INTEGER,
    v_nombre_producto TEXT,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_reg RECORD;
BEGIN
    FOR v_reg IN
        SELECT lp.linea_id,
               p.nombre AS nombre_producto,
               lp.cantidad * lp.precio_unit AS total_linea
        FROM tienda.lineas_pedido lp
        JOIN tienda.productos p ON p.producto_id = lp.producto_id
    LOOP
        v_linea_id        := v_reg.linea_id;
        v_nombre_producto := v_reg.nombre_producto;
        v_total_linea     := v_reg.total_linea;

        RETURN NEXT;
    END LOOP;
END;
$$;
```

Resumen de esta variante

Elemento	Está presente
RETURN TABLE	
RECORD	
Cursor implícito	
Transformaciones	
JOIN opcional	
SQL dinámico	(puedo añadírtelo si quieres)